FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Métodos de Otimização para o Dimensionamento de uma Viga Soldada

Diogo José Domingues Morais
José Miguel Ferreira Morete da Silva Raposo
Nuno Miguel Ferreira Rodrigues Pinto

Mestrado em Engenharia Mecânica (M.EM)

Orientador: Carlos Alberto Conceição António

Unidade Curricular: Otimização de Sistemas Mecânicos

2025-26

Resumo

O presente trabalho aborda a resolução do problema SMA7, centrado no dimensionamento ótimo de uma viga soldada sujeita a múltiplas restrições estruturais. O objetivo principal consiste em minimizar o custo total de fabrico, considerando simultaneamente os limites de tensão, deflexão, encurvadura e as condições geométricas impostas ao sistema. O estudo compara duas abordagens de otimização distintas, métodos clássicos baseados em gradiente e algoritmos genéticos, avaliando-as quanto à precisão, eficiência e robustez.

A formulação matemática do problema envolve uma função objetivo não linear e quinze restrições, tendo sido aplicada uma normalização das restrições de forma a promover um processo de convergência mais estável. No âmbito dos métodos clássicos, recorreu-se ao Lagrangeano Aumentado aliado ao Método do Gradiente Conjugado, analisando as variantes de Fletcher-Reeves e Polak-Ribière. Ambas as versões convergiram para a solução ótima teórica, tendo a abordagem de Fletcher-Reeves evidenciado maior eficiência computacional.

Paralelamente, foi implementado um Algoritmo Genético com codificação binária, elitismo, seleção SUS, cruzamento uniforme e mutação *flip-bit*. Foram ainda testados diferentes regimes de penalização dinâmica, bem como diversos níveis de precisão na codificação. Os resultados demonstram que o Algoritmo Genético consegue igualmente aproximar-se da solução ótima, embora apresente pequenas violações residuais e um custo computacional superior, consequência natural da natureza populacional e estocástica do método.

A análise comparativa final evidencia que os métodos clássicos se revelam mais rápidos e exatos quando aplicados a superfícies bem comportadas e diferenciáveis. Por sua vez, os algoritmos genéticos demonstram maior robustez em problemas de elevada complexidade e com potencial multimodalidade, constituindo uma alternativa eficaz em cenários onde métodos determinísticos enfrentam limitações. O trabalho evidencia, assim, a complementaridade entre abordagens determinísticas e evolucionárias na otimização de sistemas mecânicos.

Conteúdo

1 A Relevância da Otimização no Projeto Mecânico **1**

1.1 Introdução à Otimização em Engenharia 1

1.2 A Necessidade da Otimização em Sistemas Mecânicos 1

1.3 Distinção entre Métodos Clássicos e Genéticos 1

1.3.1 Métodos Clássicos de Otimização 2

1.3.2 Algoritmos Genéticos (Métodos Evolutivos) 2

2 Introdução e Abordagem ao Problema **3**

2.1 Enunciado do Problema 3

2.1.1 Restrições Impostas pelo Problema 4

2.1.2 Enquadramento Físico das Restrições no Problema 5

2.1.3 Dados Fornecidos 5

2.1.4 Solução Inicial e Solução Ótima 6

3 Aplicação de Métodos Clássicos de Otimização no Problema em Estudo **7**

3.1 Estrutura de um Problema de Otimização 7

3.2 Revisão Bibliográfica de Programação Não-Linear Não Restringida 8

3.2.1 Condições de Otimalidade 8

3.2.2 Algoritmos de Pesquisa Linear 9

3.2.3 Método do Gradiente (*Steepest-Descent Method*) 9

3.2.4 Método do Gradiente Conjugado 10

3.2.5 Método de Newton e Quasi-Newton 11

3.3 Revisão Bibliográfica de Programação Não-Linear Restringida 13

3.3.1 Método do Lagrangeano Aumentado 13

3.3.2 Normalização das Restrições 14

3.4 Implementação do Algoritmo 15

3.4.1 Normalização das Restrições do Problema 15

3.4.2 Implementação do Método do Lagrangeano Aumentado 16

3.4.3 Implementação do Método do Gradiente Conjugado 17

3.4.4 Algoritmo de Pesquisa Linear com *Box Constraints* 17

3.4.5 Estrutura e Implementação Computacional do Algoritmo 18

3.5 Discussão de Resultados 20
3.5.1 Resultados Comparativos: Fletcher-Reeves vs Polak-Ribière 20

4 Aplicação de Algoritmos Evolucionários na Resolução do Problema 23
4.1 Revisão Bibliográfica sobre métodos de pesquisa evolucionária 23
4.1.1 Terminologia Biológica Aplicada aos Algoritmos Genéticos 23
4.1.2 Estrutura do Algoritmo Genético 24
4.1.3 Representação do Espaço de Soluções 24
4.1.4 Definição do Mérito ou Aptidão 27
4.1.5 Métodos de Penalização em Algoritmos Genéticos 27
4.1.6 Operadores Genéticos de Seleção 28
4.1.7 Operadores Genéticos de Recombinação e Cruzamento 29
4.1.8 Operadores de Mutação 30
4.2 Implementação do Algoritmo Genético 31
4.2.1 Definição do mérito 32
4.2.2 Estratégias Elitistas 34
4.2.3 Critérios de Paragem 34
4.3 Resultados 35
4.3.1 Resultados por cenário de λ 35

5 Comparação de Resultados 45
5.1 Discussão dos Resultados 45

6 Conclusão 47

Lista de Figuras

2.1 Sistema de Viga Soldada 3

3.1 Evolução da função objetivo $f(x)$ para as duas variantes do Método do Gradiente Conjugado. 21
3.2 Evolução das variáveis de projeto para β_{FR} 22
3.3 Evolução das variáveis de projeto para β_{PR} 22

4.1 Codificação binária: cada variável x_i é representada por um bloco de l_i *bits*. . . . 26
4.2 Exemplo de Seleção através do Método *Stochastic Universal Sampling* (SUS). . . 28
4.3 Representação de Sequências Binárias e Padrões Correspondentes. 30
4.4 O Operador de Mutação *flip-bit*. 31
4.5 Evolução de $f(x)$ do Melhor Indivíduo ao Longo das Gerações (*Run* 25) 37
4.6 Histórico do mérito ao longo das gerações, comparando o melhor indivíduo com o mínimo da elite. 37
4.7 Penalização do melhor indivíduo e evolução de λ ao longo das gerações (método dinâmico). 37
4.8 Evolução das variáveis do melhor indivíduo (*Run* 25): x_1 (espessura h), x_2 (comprimento l), x_3 (largura t) e x_4 (altura b). 38
4.9 Evolução da entropia (Shannon) por variável ao longo das gerações (Run 25), evidenciando perda de diversidade após a convergência. 38
4.10 Distribuição da população e da elite em diferentes gerações (*Run* 25) 39
4.11 Comparação da evolução do mérito do melhor indivíduo e da elite para diferentes precisões (2 vs 5 casas decimais). 40
4.12 Comparação da evolução da entropia (Shannon) por variável para diferentes precisões (2 vs 5 casas decimais). 41
4.13 Evolução do mérito ao longo das gerações (método de Joines et al.). 42
4.14 Evolução da função objetivo $f(x)$ do melhor indivíduo (método de Joines et al.). 42
4.15 Penalização do melhor indivíduo e evolução do parâmetro $(Ck)^{\alpha}$ ao longo das gerações (método de Joines et al.). 43
4.16 Evolução da entropia (Shannon) por variável ao longo das gerações (método de Joines et al.). 43

Lista de Tabelas

2.1 Enquadramento Físico das Restrições 5
2.2 Propriedades Fornecidas pelo Problema 5

3.1 Resultados do Método do Lagrangeano Aumentado para Polak-Ribière (β_{PR}) . . 20
3.2 Resultados do Método do Lagrangeano Aumentado para Fletcher-Reeves (β_{FR}) . 21

4.1 Parâmetros Fundamentais do Algoritmo Genético 32
4.2 Constantes Utilizadas na Função de Penalização Dinâmica (Método de Joines et al.) 33
4.3 Resumo dos Melhores Resultados por Cenário de λ. 35
4.4 Impacto da precisão na melhor execução para $\lambda = 20$. 40
4.5 Comparação entre os métodos de penalização dinâmica (Joines et al.) e Hadj-Alouane & Bean. 42

5.1 Comparação entre métodos de otimização. 45

Capítulo 1

A Relevância da Otimização no Projeto Mecânico

1.1 Introdução à Otimização em Engenharia

A otimização é, fundamentalmente, o ato de obter o melhor resultado possível sob um determinado conjunto de circunstâncias. No projeto, construção e manutenção de qualquer sistema de engenharia, os engenheiros necessitam de tomar múltiplas decisões tecnológicas e de gestão em várias etapas do processo. O objetivo final de todas essas decisões é minimizar o esforço necessário (como custo, peso ou energia) ou maximizar o benefício desejado (como eficiência, resistência ou durabilidade).

Visto que o esforço necessário ou o benefício desejado em qualquer situação prática pode ser expresso como uma função de certas variáveis de decisão, a otimização define-se como o processo matemático e lógico de encontrar as condições que resultam no valor máximo ou mínimo de uma função [1, 5].

1.2 A Necessidade da Otimização em Sistemas Mecânicos

Em sistemas mecânicos, a otimização não é apenas uma questão de redução de custos, mas também de garantia de integridade estrutural sob restrições físicas rigorosas.

O processo de otimização pode-se tornar complexo devido à presença de restrições de comportamento. Assim, a relevância da otimização reside na capacidade de navegar em um espaço de design complexo para encontrar o equilíbrio ideal entre economia, $f(x)$, e segurança (restrições $g(x)$ e $h(x)$).

1.3 Distinção entre Métodos Clássicos e Genéticos

Nesta unidade curricular abordamos duas grandes classes de algoritmos que podem ser usadas para resolver problemas de otimização e que serão usadas neste relatório. São eles os **Métodos**

Clássicos e os **Algoritmos Genéticos (Evolutivos)**. A escolha entre eles depende da natureza da função objetivo e das restrições.

1.3.1 Métodos Clássicos de Otimização

Os métodos clássicos, também conhecidos como métodos baseados em gradiente ou determinísticos, utilizam o cálculo diferencial para determinar a direção de busca.

- **Abordagem**: Geralmente, partem de um ponto inicial (x^{start}) e movem-se iterativamente na direção onde a função objetivo decresce mais rapidamente (baseado no gradiente ∇f).
- **Vantagens**: São matematicamente rigorosos e tendem a convergir rapidamente para uma solução muito precisa.
- **Desvantagens**: O seu principal defeito é a tendência para ficarem "presos"em ótimos locais. Se a superfície da função objetivo tiver vários "vales", um método clássico irá convergir para o fundo do vale mais próximo do ponto inicial, ignorando a existência de um vale mais profundo (o ótimo global) noutra parte do espaço de procura. Além disso, exigem que as funções sejam contínuas e deriváveis [5].

1.3.2 Algoritmos Genéticos (Métodos Evolutivos)

Os algoritmos genéticos são métodos de busca estocástica baseados nos princípios da seleção natural e genética biológica. Ao contrário dos métodos clássicos que trabalham com um único ponto, estes trabalham com uma população de soluções candidatas.

- **Abordagem**: A cada geração, a população evolui através de operadores de seleção, cruzamento (crossover) e mutação. As melhores soluções (indivíduos com menor custo $f(x)$) têm maior probabilidade de gerar descendentes.
- **Vantagens**: Devido à sua natureza populacional e ao uso de mutação, são excelentes na exploração global do espaço de procura, sendo muito menos propensos a ficar presos em ótimos locais. Não necessitam de derivadas, o que os torna ideais para funções complexas ou descontínuas.
- **Desvantagens**: São computacionalmente mais pesados e lentos, pois requerem muitas avaliações da função objetivo. Além disso, não garantem a solução ótima exata da solução final, mas sim uma estimativa muito próxima [1, 5].

Capítulo 2

Introdução e Abordagem ao Problema

Este capítulo apresenta o problema em estudo neste projeto, o problema **SMA 7**, que consiste no **Dimensionamento de uma Viga Soldada**, bem como todas as restrições associadas ao mesmo e ainda a metodologia adotada para se ober uma solução otimizada.

2.1 Enunciado do Problema

O problema em estudo prende-se no dimensionamento da viga presente na Figura 2.1, minimizando o máximo possível o custo global da estrutura. Este projeto é limitado por um conjunto de restrições que incluem: a tensão de corte no cordão de soldadura (τ), a tensão de flexão na viga (σ), a carga crítica de encurvadura na barra (P_c), a deflexão na extremidade da viga (δ) e ainda restrições adicionais. O vetor das variáveis de projeto é ainda definido por: $\{x_1, x_2, x_3, x_4\} = \{h, l, t, b\}$.

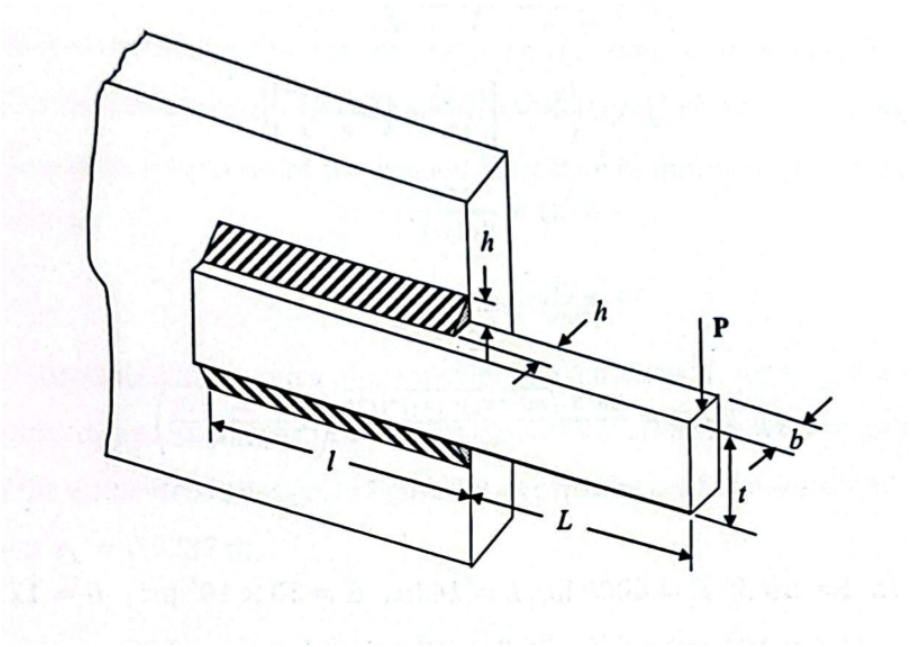


Figura 2.1: Sistema de Viga Soldada

Em que **h** representa a altura e largura da solda, **l** o comprimento da solda, **t** o comprimento da viga e **b** a largura da própria viga.

Através da análise geométrica do problema, é possível definir a função objetivo, tal como mostra a equação 2.1:

$$f(x) = 1.10471(x_1)^2 x_2 + 0.04811 x_3 . x_4 (14.0 + x_2) \tag{2.1}$$

A função objetivo, $f(x)$, representa assim o custo total de fabrico, composto pelo custo do material de adição da soldadura (termo dependente de x_1 e x_2) e pelo custo do material da barra (termo dependente de x_3 e x_4).

2.1.1 Restrições Impostas pelo Problema

Tal como foi mencionado, o problema impõe ainda um total de quinze restrições adicionais, dadas pelas seguintes equações:

$$g_1(x) = \tau(x) - \tau_{max} \leq 0 \tag{2.2}$$

$$g_2(x) = \sigma(x) - \sigma_{max} \leq 0 \tag{2.3}$$

$$g_3(x) = x_1 - x_4 \leq 0 \tag{2.4}$$

$$g_4(x) = 0.10471(x_1)^2 + 0.04811 x_3 x_4 (14.0 + x_2) - 5.0 \leq 0 \tag{2.5}$$

$$g_5(x) = 0.125 - x_1 \leq 0 \tag{2.6}$$

$$g_6(x) = \delta(x) - \delta_{max} \leq 0 \tag{2.7}$$

$$g_7(x) = P - P_c(x) \leq 0 \tag{2.8}$$

$$g_8(x) \text{ a } g_{11}(x) : 0.1 \leq x_i \leq 2.0, \quad i = 1,4 \tag{2.9}$$

$$g_{12}(x) \text{ a } g_{15}(x) : 0.1 \leq x_i \leq 10.0, \quad i = 2,3 \tag{2.10}$$

Para o cálculo destas restrições são então estabelecidas relações auxiliares, dadas pelas seguintes funções:

$$\tau(x) = \sqrt{(\tau')^2 + 2\tau'\tau''\frac{x_2}{2R} + (\tau'')^2} \tag{2.11}$$

$$\sigma(x) = \frac{6PL}{x_4(x_3)^2} \tag{2.12}$$

$$\delta(x) = \frac{4PL^3}{Ex_4(x_3)^3} \tag{2.13}$$

$$P_c(x) = \frac{4.013\sqrt{EG[(x_3)^2(x_4)^6]/36}}{L^2}\left(1 - \frac{x_3}{2L}\sqrt{\frac{E}{4G}}\right) \tag{2.14}$$

Para a análise das tensões na soldadura, considera-se a sobreposição dos efeitos de corte direto e de torção no plano da solda. Assim, τ' representa a tensão de corte primária e τ'' a tensão de

corte secundária (devida ao momento), calculadas através das propriedades geométricas da secção soldada, onde M é o momento fletor aplicado no centroide da solda, R é a distância radial ao ponto crítico e J é o momento polar de inércia do grupo de solda:

$$\begin{cases} \tau' = \frac{P}{\sqrt{2}x_1x_2} \\ \tau'' = \frac{MR}{J} \end{cases} \quad e \quad \begin{cases} R = \sqrt{\frac{(x_2)^2}{4} + \left(\frac{x_1+x_3}{2}\right)^2} \\ M = P\left(L + \frac{x_2}{2}\right) \\ J = 2\left\{\frac{x_1x_2}{\sqrt{2}}\left[\frac{(x_2)^2}{12} + \left(\frac{x_1+x_3}{2}\right)^2\right]\right\} \end{cases} \tag{2.15}$$

P, M, J também são variáveis que dependem das variáveis de projeto: x_1, x_2 *e* x_3.

2.1.2 Enquadramento Físico das Restrições no Problema

Com base nas restrições abordadas na Secção 2.1.1, é possível fazer a analogia entre as restrições definidas e o seu enquadramento físico no problema. A Tabela 2.1 contêm todas as restrições e o seu enquadramento físico no problema.

Tabela 2.1: Enquadramento Físico das Restrições

$g_1(x)$	A tensão de corte no cordão não pode exceder a tensão de corte máxima (τ_{max})
$g_2(x)$	A tensão de flexão da viga não pode exceder a tensão de flexão máxima (σ_{max})
$g_3(x)$	A largura da solda (h) não pode exceder a largura da viga (b)
$g_4(x)$	O limite superior no custo máximo permitido para o projeto da viga soldada
$g_5(x)$	A largura da solda (h) deve ser superior a 0.125 in.
$g_6(x)$	A deflexão da viga não pode exceder a deflexão máxima (δ_{max})
$g_7(x)$	A carga aplicada na viga não pode exceder a carga crítica de encurvadura (P_c)
$g_8(x)$ a $g_{11}(x)$	Valores mínimos e máximos da largura de solda (h) e da largura da viga (b)
$g_{12}(x)$ a $g_{15}(x)$	Valores mínimos e máximos do comprimento de solda (l) e da altura da viga (t)

2.1.3 Dados Fornecidos

Além destas restrições e dos dados até agora mencionados, o problema fornece ainda alguns valores dados como constantes para algumas propriedades já abordadas. Estes valores estão definidos na Tabela 2.2.

Tabela 2.2: Propriedades Fornecidas pelo Problema

Força, P (lb)	6000
Comprimento, L (in.)	14
Módulo de Young, E (psi)	30 x 10^6
Módulo de Elasticidade ao Corte, G (psi)	12 x 10^6
Tensão de Corte Máxima, τ_{max} (psi)	13.6 x 10^3
Tensão de Flexão Máxima, σ_{max} (psi)	30.0 x 10^3
Deflexão Máxima, δ_{max} (in.)	0.25

Como este problema está formulado no Sistema Imperial (in, lb, psi), a sua solução também será apresentada nesse mesmo sistema.

2.1.4 Solução Inicial e Solução Ótima

Por fim, o problema define ainda os valores da função objetivo e do vetor das variáveis de projeto, para a solução inicial e para a solução ótima, utilizando os sobreescritos "*start*" e *, respetivamente.

$$x^{start} = \{h, l, t, b\} = \{0.4, 6.0, 9.0, 0.5\} \text{ in.} \rightarrow f^{start} = 5.3904,$$

$$x^{*} = \{h, l, t, b\} = \{0.2444, 6.2177, 8.2915, 0.2444\} \text{ in.} \rightarrow f^{*} = 2.3810$$

Capítulo 3

Aplicação de Métodos Clássicos de Otimização no Problema em Estudo

Neste capítulo, aborda-se inicialmente a estrutura adotada para a formulação do problema de otimização, seguida de uma revisão bibliográfica sobre os métodos clássicos de otimização, bem como a sua aplicação prática na resolução do problema em estudo.

3.1 Estrutura de um Problema de Otimização

Geralmente, a resolução de um problema de otimização segue uma estrutura metodológica bem definida, de forma a organizar e compreender o processo de resolução de sistemas complexos. Essa estrutura é conhecida como *"Three-Columns Concept"* (Conceito das Três Colunas) e divide-se, tal como o nome indica, em três partes essenciais para qualquer sistema de otimização iterativo.

- **Formulação e Modelagem do Sistema:** Consiste na definição matemática rigorosa do problema e na representação do seu contexto físico. Nesta fase, estabelece-se a definição da:
  - **Função Objetivo -** $f(x)$**:** define o valor que se pretende minimizar.
  - **Restrições -** $g(x)$ **e** $h(x)$**:** definem os limites físicos e geométricos que a solução deve respeitar.

  Esta etapa, descrita no Capítulo 2, tem a função de fornecer a base analítica para a otimização, delimitando o espaço de procura e a superfície a ser navegada.

- **Algoritmo de Otimização:** Realiza a busca iterativa para encontrar o mínimo da função definida pelo modelo. Corresponde ao **ciclo interno** do código.

  Partindo de uma solução inicial, o algoritmo produz uma sequência de soluções melhoradas (no nosso caso usando o **Método do Gradiente Conjugado**). O processo iterativo interno ocorre até se encontrar um mínimo local para o subproblema atual, aproximando-se progressivamente das condições de otimalidade de *Karush-Kuhn-Tucker (KKT)*.

- **Modelo de Otimização:** Corresponde ao **ciclo externo** do algoritmo. Estabelece a ponte entre a formulação física e o algoritmo numérico, transformando o problema original (com restrições) num formato que possa ser resolvido matematicamente.

  No contexto deste projeto, esta fase refere-se à construção da **Função Lagrangeana Aumentada**. É responsável por controlar a evolução do sistema, garantindo que as restrições são gradualmente satisfeitas através do ajuste dos parâmetros de penalidade (ε) e dos multiplicadores de Lagrange (λ), fornecendo ainda o critério final de paragem.

3.2 Revisão Bibliográfica de Programação Não-Linear Não Restringida

A **Otimização Não-Linear Não Restringida** é a base fundamental para a resolução da maioria dos problemas de otimização contínua. Neste caso, a natureza não-linear da função objetivo (Equação (2.1)) e também das restrições do problema justificam a aplicabilidade de métodos de otimização não lineares como aqueles que vão ser abordados nesta secção.

Os modelos abordados nesta secção geram uma sequência (x^k), que começa em x^0, e segue a fórmula iterativa dada na Equação 3.1.

$$x^{k+1} = x^k + \alpha^k d^k \tag{3.1}$$

A direção d^k representa a direção de pesquisa e α^k corresponde à amplitude do passo ao longo dessa direção. A distinção entre diferentes métodos de otimização reside na forma como d^k e α^k são determinados. A escolha da direção de pesquisa influencia o comportamento local do algoritmo e a sua taxa de convergência, enquanto a convergência global está frequentemente associada à seleção adequada do tamanho do passo α^k.

O propósito de considerar diferentes estratégias para a determinação de α^k é garantir que o algoritmo definido por (3.1) seja globalmente convergente, idealmente sem comprometer a taxa de convergência.

3.2.1 Condições de Otimalidade

Matematicamente, o problema define-se como a procura de um vetor de variáveis de decisão $x^* \in \mathbb{R}^n$ que minimize uma função objetivo escalar $f(x)$:

$$\min_{x \in \mathbb{R}^n} f(x)$$

Por definição, para a solução de um problema de otimização, o objetivo é obter o ponto mínimo global de $f(x)$ dentro do domínio da função. Porém, em cenários reais de engenharia, muitas das vezes já é bastante satisfatório obter o ponto que corresponde ao **mínimo relativo ou local**.

Para garantir que um ponto x^* é um mínimo relativo, recorre-se à expansão em Série de Taylor. As condições dividem-se em:

- **Condição Necessária de 1ª Ordem:** Se x^* é um minimizante local e f é continuamente diferenciável, então o gradiente deve ser nulo:

$$\nabla f(x^*) = 0 \tag{3.2}$$

- **Condição Suficiente de 2ª Ordem:** Se $\nabla f(x^*) = 0$ e a matriz Hessiana, $H_f(x^*) = \nabla^2 f(x^*)$, for definida positiva, então x^* é um minimizante local estrito:

$$H_f(x^*) > 0 \tag{3.3}$$

3.2.2 Algoritmos de Pesquisa Linear

Os algoritmos de Pesquisa Linear baseiam-se na determinação da amplitude α^k ao longo da direção d^k, de forma a garantir a convergência da fórmula iterativa apresentada na Equação 3.1.

Para calcular α^k, é comum recorrer ao **Algoritmo de Pesquisa Linear de Armijo**. Este método impõe que o passo α^k proporcione uma redução suficiente na função objetivo, garantindo que a diminuição obtida seja proporcional à inclinação dada pelo gradiente.

$$f(x^k + \alpha d^k) \leq f(x^k) + \gamma\, \alpha\, \nabla f(x^k)^T d^k$$

Isto evita passos demasiado grandes que fariam o algoritmo oscilar ou divergir. A sua forma geral é a seguinte [1]:

Passo 1: Escolher Δ^k, tal que:

$$\Delta^k \geq \frac{\left|\nabla f(x^k)^T d^k\right|}{||d^k||^2}, \text{ definindo } \alpha = \Delta^k.$$

Passo 2: Se: $f(x^k + \alpha d^k) \leq f(x^k) + \gamma\, \alpha\, \nabla f(x^k)^T d^k$ então:

$$\alpha^k = \alpha$$

Parar.

Passo 3: Senão, se:

defina $\alpha \leftarrow \delta\alpha$

E vai para o Passo 2.

3.2.3 Método do Gradiente (*Steepest-Descent Method*)

Ao utilizar o método da Secção 3.2.2 para calcular α^k, fica ainda a necessidade de calcular a direção da pesquisa d^k, ou seja, a direção que cada iteração vai seguir para no final se obter a

solução mais próxima da solução ótima.

Para isso, é importante inicialmente introduzir o método do gradiente (*steepest-descent method*), define a direção de pesquisa como o negativo do gradiente:

$$d^k = -\nabla f(x^k)$$

O algoritmo geral deste método é o seguinte [1]:

Passo 1: Escolha $x^0 \in \mathbb{R}^n$ e faça com que $k = 0$.

Passo 2: Se $\nabla f(x^k) = 0$, então:

Para.
Fim da condição "Se".

Passo 3: Faça: $d^k = -\nabla f(x^k)$ e encontre α^k que minimiza a função f na direção de d^k. Para o efeito de pesquisa linear, use o algoritmo de Armijo apresentado na Secção 3.2.2, ou outra técnica de minimização com uma variável.

Passo 4: Faça:

$$k^{k+1} = x^k - \alpha^k \nabla f(x^k)$$
$$k \leftarrow k+1$$

Seguir para o **Passo 2**.

Apesar de ser um bom ponto de partida, este método não é satisfatório do ponto de vista prático, dado que a sua **taxa de convergência é normalmente lenta** perto do ótimo, exibindo um comportamento de "ziguezague", especialmente em funções com vales estreitos e alongados.

3.2.4 Método do Gradiente Conjugado

Através do metódo explicado na Secção 3.2.3, surgiu um novo que tinha o objetivo de acelerar o método do gradiente e que, originalmente pretendia resolver a minimização de funções quadráticas estritamente convexas, este método designa-se por Algoritmo das Direções Conjugadas (ADC) e utiliza como dados de entrada as direções d^0 até d^{n-1}.

O método que será utilizado nesta parte do problema é, no entanto, o método do gradiente conjugado (*Conjugate Gradient Algorithm*), cujas direções, ao contrário do ADC não são dadas, mas sim, geradas iterativamente através da seguinte regra:

$$\mathbf{d}^k = \begin{cases} -\nabla f(\mathbf{x}^k) & , \quad k = 0 \\ -\nabla f(\mathbf{x}^k) + \beta^{k-1}\mathbf{d}^{k-1} & , \quad k \geq 1 \end{cases}$$

O escalar β^k é escolhido de forma a impôr a condição de direções conjugadas e as formas mais comuns para a sua escolha passam pelas fórmulas de Fletcher-Reeves e de Polak-Ribière, presentes nas Equações 3.4 e 3.5, respetivamente.

$$\beta_{FR}^{k} = \frac{||\nabla f(x^{k+1}||^2}{||\nabla f(x^k)||^2} \tag{3.4}$$

$$\beta_{PR}^{k} = \frac{\nabla f(x^{k+1})^T(\nabla f(x^{k+1}) - \nabla f(x^k))}{||\nabla f(x^k)||^2} \tag{3.5}$$

O algoritmo geral deste método é o seguinte [1]:

Passo 1: Escolha $x^0 \in \mathbb{R}^n$ e faça com que $k = 0$.

Passo 2: Se $\nabla f(x^k) = 0$, então:

Parar.

Fim da condição "Se".

Passo 3: Calcula β^{k-1} através de 3.4 ou 3.5 e escolha a direção, tal que:

$$\mathbf{d}^k = \begin{cases} -\nabla f(\mathbf{x}^k) & , \quad k = 0 \\ -\nabla f(\mathbf{x}^k) + \beta^{k-1}\mathbf{d}^{k-1} & , \quad k \geq 1 \end{cases}$$

Passo 4: Encontrar α^k que minimiza a função f na direção d^k. Pode ser utilizado o algoritmo de Armijo ou outra técnica de minimização a uma variável.

Passo 5: Faça:

$$x^{k+1} = x^k + \alpha^k d^k,$$

$$k \leftarrow k+1$$

Seguir para o **Passo 2**.

Fim do ciclo "Faça".

3.2.5 Método de Newton e Quasi-Newton

O Método de Newton baseia-se na ideia de aproximar a função objetivo $f(x)$ localmente por uma função quadrática, usando a Série de Taylor de segunda ordem. Assim, ao invés de usar apenas a **inclinação** (gradiente) para decidir a direção, o método usa a **curvatura** da função. A curvatura é dada pela matriz Hessiana, $H_f(x^*) = \nabla^2 f(x^*)$.

A fórmula de atualização é:

$$x^{k+1} = x^k - [H_f(x^k)]^{-1}\nabla f(x^k)$$

A principal vantagem do Método de Newton é a sua **convergência quadrática**. Quando a solução inicial se encontra suficientemente próxima do ponto ótimo, o método atinge a solução em muito poucas iterações, tornando-se significativamente mais rápido do que qualquer método baseado apenas no gradiente.

No entanto, aplicar o Método de Newton na sua forma pura exige o cálculo explícito da matriz Hessiana de dimensão 4×4. Considerando que a função objetivo $f(x)$ não é um polinómio simples e que envolve restrições adicionais, como a carga crítica de encurvadura (P_c) e a tensão de corte (τ), ambas dependentes de expressões com raízes, potências e divisões, o cálculo analítico das segundas derivadas torna-se **extremamente trabalhoso** e altamente **suscetível a erros humanos**.

Além disso, mesmo que a Hessiana fosse obtida, surgiria ainda a questão da sua *definitude*. Caso a matriz Hessiana não fosse positiva definida, situação comum em regiões de curvatura negativa, como pontos de sela, o Método de Newton poderia falhar, divergir ou ser atraído para um ponto crítico que não corresponde a um mínimo. Embora existam técnicas para forçar a Hessiana a ser positiva definida, tais abordagens aumentariam significativamente a complexidade computacional do método.

Face a estas limitações, o Método Quasi-Newton surge como uma alternativa prática e eficiente. Este método procura **preservar a rapidez de convergência do Método de Newton, evitando, contudo, o custo computacional de calcular e inverter a Hessiana exata a cada iteração**. Ao invés disso, constrói-se iterativamente uma aproximação da inversa da Hessiana, $B_k \approx [\nabla^2 f(x_k)]^{-1}$, atualizada com base apenas nas variações observadas no gradiente entre iterações consecutivas. Desta forma, alcança-se um equilíbrio entre eficiência computacional e rapidez de convergência, tornando o Método Quasi-Newton especialmente adequado para problemas não lineares de maior complexidade.

A fórmula geral é:

$$x^{k+1} = x^k - B_k \nabla f(x^k)$$

Embora o Método Quasi-Newton apresente a vantagem de proporcionar uma convergência muito eficiente, significativamente superior ao Gradiente Conjugado e próxima da obtida pelo Método de Newton, sem necessidade de calcular derivadas de segunda ordem, este método implica armazenar e atualizar uma matriz densa de dimensão $N \times N$. Tal matriz constitui uma aproximação da curvatura da função, construída a partir da informação recolhida ao longo das iterações. Contudo, como a função objetivo sofre alterações substanciais de uma iteração para a seguinte (como será demonstrado mais adiante), a matriz aproximada do Quasi-Newton pode rapidamente deixar de representar adequadamente a curvatura local. Nestas circunstâncias, seria necessário reinicializar frequentemente essa matriz, anulando assim a principal vantagem do método, nomeadamente a sua **rapidez e eficiência computacional**.

3.3 Revisão Bibliográfica de Programação Não-Linear Restringida

Concluída a análise dos métodos de minimização não restringida, torna-se fundamental abordar a otimização restringida, um cenário muito mais comum no dimensionamento de sistemas mecânicos. Nestes casos, o domínio de pesquisa deixa de ser todo o espaço $\mathbb{R}^n$, ficando circunscrito a uma região de admissibilidade definida por requisitos físicos e operacionais (restrições). A presença destas restrições obriga a que os algoritmos de pesquisa não só minimizem a função objetivo, mas garantam estritamente a viabilidade da solução. Pelo que, a forma geral do problema da programação não-linear restringida é a seguinte:

$$\min_{\mathbf{x}\in\mathbf{X}} f(x),$$

Onde $x \in \mathbb{R}^n$ é o vetor de variáveis de projeto, $f : \mathbb{R}^n \to \mathbb{R}$ é a função objetivo e o espaço admissível (domínio de projeto) $\mathbf{X}$ é descrito pelas restrições de igualdade e/ou desigualdade impostas nas variáveis de projeto, tal que:

$$\mathbf{X} = \{\mathbf{x} \in \mathbb{R}^n : g_i(\mathbf{x}) \leq 0, i = 1, \ldots, p; h_j(\mathbf{x}) = 0, j = 1, \ldots, m, m \leq n\}$$

Pode-se então considerar os métodos para resolver problemas de programação não-linear restringida como:

$$\mathrm{Min}_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x})$$

Sujeito a,

$$g(\mathbf{x}) \leq \mathbf{0}\ h(\mathbf{x}) = 0,$$

onde, $g : \mathbb{R}^n \to \mathbb{R}^p$ e $h : \mathbb{R}^n \to \mathbb{R}^m$, com $m \leq n$.

3.3.1 Método do Lagrangeano Aumentado

O metódo do Lagrangeano aumentado foi inicialmente desenvolvido para problemas com restrições de igualdade e, mais tarde, extendido para problemas com restrições de desigualdade.

Observando primeiramente este método para problemas com restrições de igualdade temos que: $\mathrm{Min}_{\mathbf{x}\in\mathbf{X}}\ f(x)$, sujeito a $\mathrm{h}(\mathbf{x}) = \mathbf{0}$.

A função Lagrangeana para este problema é dada pela Equação 3.6:

$$L(\mathbf{x}, \mu) = f(\mathbf{x}) + \mu^T \mathrm{h}(\mathbf{x}) \tag{3.6}$$

Sendo que, a função Lagrangeana aumentada é dada pela Equação 3.7:

$$L_a(\mathbf{x}, \mu; \varepsilon) = \mathbf{L}(\mathbf{x}, \mu) + \frac{\mathbf{1}}{\varepsilon}||\mathrm{h}(\mathbf{x}||^2 \tag{3.7}$$

Posto isto, é importante assumir a possibilidade de existência de restrições de desigualdade, do tipo $g(\mathbf{x}) \leq 0$, tal como as que existem no problema abordado neste relatório. Neste caso,

é possível reescrever a inequação de modo a convertê-la numa restrição de igualdade do tipo, $g_i(\mathbf{x}) + y_i^2 = 0$, onde y_i^2 é uma variavél adicional de folga. Convém notar que esta transformação é apenas instrumental e que tem como desvantagem o aumento do número de variáveis.

Seja λ_i o multiplicador associado à restrição g_i, então a função Lagrangeana aumentada para o problema de restrições de desigualdade reescrito numa igualdade é dado pela equação 3.8:

$$L_a(\mathbf{x},\mathbf{y},\lambda;\varepsilon) = f(\mathbf{x}) + \sum_{i=1}^{p} \lambda_i \left[g_i(\mathbf{x}) + y_i^2\right] + \frac{1}{\varepsilon}\sum_{i=1}^{p}\left[g_i(\mathbf{x}) + y_i^2\right]^2 \tag{3.8}$$

A função $L_a(\mathbf{x},\mathbf{y},\lambda;\varepsilon)$, por sua vez, pode ser minimizada em relação a x e y para valores dados de λ e de ε. No entanto, para valores dados de x, λ e ε, o mínimo y_i^2 de L_a é dado por $y_i^2(x,\lambda,\varepsilon) = -\min\left\{0, g_i(x) + \frac{\varepsilon}{2}\lambda_i\right\}$. Pelo que, ao substituir este fator na equação 3.8, obtêm-se:

$$L_a(\mathbf{x},\lambda;\varepsilon) = f(\mathbf{x}) + \lambda^T \max\left\{\mathbf{g}(\mathbf{x}), -\frac{\varepsilon}{2}\lambda\right\} + \frac{1}{\varepsilon}\left\|\max\left\{\mathbf{g}(\mathbf{x}), -\frac{\varepsilon}{2}\lambda\right\}\right\|^2 \tag{3.9}$$

Tal como já foi abordado, dada a natureza das restrições impostas pelo problema em estudo, cujas restrições são todas de desigualdade, esta função é aquela que vai ser minimizada.

Quanto ao algoritmo do método Lagrangeano aumentado, este é dado por:

Passo 1: Escolher ε^0, μ^0 e $\mathbf{x}^s \in \mathbb{R}^n$; definir $k = 0$.

Passo 2: Partindo de $\mathbf{x}^s$, determinar um mínimo local sem restrições

$$\mathbf{x}^k = \arg\min_{\mathbf{x}\in\mathbb{R}^n} L_a(\mathbf{x},\mu^k;\varepsilon^k).$$

Passo 3: Se $(\mathbf{x}^k,\mu^k)$ for um par Karush-Kuhn-Tucker, então:

parar.
Fim da condição "Se".

Passo 4: Escolher um parâmetro de penalidade $\varepsilon^{k+1} \in (0,\varepsilon^k]$.

Passo 5: Atualizar a estimativa do multiplicador para μ^{k+1}.

Passo 6: Faça

$$\mathbf{x}^s = \mathbf{x}^k,$$
$$k \leftarrow k+1$$

e volta ao **Passo 2**.

3.3.2 Normalização das Restrições

Nos cálculos numéricos aplicados à otimização de engenharia, é muito importante normalizar todas as funções de restrição. As restrições ativas são fundamentais para definir a direção de pesquisa, sendo comum a utilização de um valor único para o parâmetro de penalidade (tipicamente

$\varepsilon \approx 0.10$) transversal a todas as restrições. No entanto, uma vez que diferentes restrições podem envolver diferentes ordens de grandeza, a utilização de um ε uniforme torna-se inadequada se as restrições não estiverem normalizadas.

Considere-se, a título de exemplo, uma restrição de tensão e uma restrição de deslocamento, formuladas respetivamente como:

$$\sigma \leq \sigma_a \quad \text{ou} \quad \sigma - \sigma_a \leq 0 \tag{3.10}$$

e

$$\delta \leq \delta_a \quad \text{ou} \quad \delta - \delta_a \leq 0, \tag{3.11}$$

onde σ e δ representam a tensão e o deslocamento calculados num ponto, e σ_a e δ_a os respetivos valores admissíveis. É evidente que as unidades das duas restrições são distintas: a primeira envolve tensão (Pascal, N/m^2), enquanto que a segunda refere-se ao deslocamento da estrutura, que pode ser da ordem de apenas alguns centímetros ou centésimos de metro. Consequentemente, os valores numéricos destas restrições possuem ordens de grandeza muito diferentes. Se as restrições forem violadas, torna-se difícil avaliar a severidade relativa dessa violação ou garantir uma convergência equilibrada do algoritmo.

Para resolver este problema, procede-se à normalização das restrições dividindo-as pelos seus valores admissíveis. Desta forma, obtêm-se as formas normalizadas adimensionais:

$$\frac{\sigma}{\sigma_a} - 1 \leq 0 \quad \text{e} \quad \frac{\delta}{\delta_a} - 1 \leq 0. \tag{3.12}$$

Assume-se aqui que os valores admissíveis (σ_a, δ_a) são positivos; caso contrário, o sentido das desigualdades ter-se-ia de inverter. Este procedimento garante que as restrições operam numa escala numérica semelhante, facilitando o processo de otimização.

Importa notar, contudo, que existem algumas restrições que não podem ser normalizadas através deste método. Para estes casos, o valor admissível pode ser zero. Por exemplo, o limite inferior de algumas variáveis de projeto pode ser nulo, impedindo a divisão pelo valor de referência. Tais restrições devem, portanto, ser mantidas e tratadas na sua forma original.

3.4 Implementação do Algoritmo

A seguinte secção apresenta de forma detalhada os principais aspetos do algoritmo desenvolvido para a resolução do problema em estudo, bem como os resultados obtidos e a explicação dos mesmos.

3.4.1 Normalização das Restrições do Problema

Tal como foi referido na Secção 3.3.2, este é um passo importante para garantir que as restrições operam numa escala numérica semelhante. Convém notar, no entanto, que este passo de

normalização das restrições apenas se aplica a restrições que não sejam restrições de tamanho, todas as outras são tratadas como *box constraints*, e por isso não necessitam de normalização.

As seguintes equações mostram então a normalização das restrições g_1, g_2, g_3, g_4, g_6 e g_7, Equações, 2.2, 2.3, 2.4, 2.5, 2.7 e 2.8, respetivamente:

$$g_1(x) = \frac{\tau}{\tau_{max}} - 1 \leq 0 \tag{3.13}$$

$$g_2(x) = \frac{\sigma}{\sigma_{max}} - 1 \leq 0 \tag{3.14}$$

$$g_3(x) = \frac{x_1}{x_4} - 1 \leq 0 \tag{3.15}$$

$$g_4(x) = \frac{0.10471(x_1)^2 + 0.04811 x_3 x_4 (14.0 + x_2)}{5.0} - 1 \leq 0 \tag{3.16}$$

$$g_6(x) = \frac{\delta(x)}{\delta_{max}} - 1 \leq 0 \tag{3.17}$$

$$g_7(x) = \frac{P}{P_c(x)} - 1 \leq 0 \tag{3.18}$$

3.4.2 Implementação do Método do Lagrangeano Aumentado

Concluída a normalização das restrições, torna-se possível implementar o método do Lagrangeano Aumentado, seguindo uma abordagem análoga à apresentada na Secção 3.3.1.

Para além do ponto inicial definido pelo enunciado do problema, é necessário estabelecer um conjunto de parâmetros iniciais. O multiplicador de Lagrange é iniciado com o valor $\varepsilon = 0.1$, enquanto o vetor λ é configurado com todas as suas componentes nulas. Tal como foi explicado, a cada iteração um novo vetor x (vetor das variáveis de projeto) é calculado e a função Lagrangeana é minimazada com os valores atuais de ε e λ, através da Equação 3.9.

Para a iteração seguinte, os valores de ε e λ são atualizados segundo as seguintes equações:

$$\lambda^{k+1} = \max\left\{0, \lambda^k + \frac{2}{\varepsilon^k} g(x^k)\right\} \tag{3.19}$$

$$\varepsilon^{k+1} = \rho \times \varepsilon^k \tag{3.20}$$

Onde, $\rho = 0.9$ foi o valor utilizado para a Equação 3.20. Quanto ao critério de paragem, utilizaram-se as condições de Karush-Kuhn-Tucker, dadas por:

$$|\nabla f(x^k) + \lambda^T \nabla g(x^k)| \leq tol \tag{3.21}$$

$$|\lambda^T g(x^k)| \leq tol \tag{3.22}$$

Onde a variável ***tol*** representa a tolerância estabelecida para as iterações, que neste caso assumiu-se como $tol = 1 \times 10^{-4}$.

3.4.3 Implementação do Método do Gradiente Conjugado

Para a resolução do subproblema de minimização inerente a cada ciclo do algoritmo do Lagrangeano Aumentado, desenvolveu-se um *script* baseado no Método do Gradiente Conjugado. Esta modularização permite que a função de otimização principal invoque o algoritmo de busca de forma eficiente para encontrar os mínimos locais da função Lagrangeana.

A estrutura lógica implementada segue os princípios clássicos do método, incorporando critérios de paragem baseados numa tolerância pré-definida e num limite máximo de iterações. Adicionalmente, o código oferece flexibilidade na computação do parâmetro de conjugação β, permitindo ao utilizador optar pelas formulações de:

- **Fletcher-Reeves:**

$$\beta_{FR}^{(k)} = \frac{\|\nabla L_a(\mathbf{x}^{(k+1)})\|^2}{\|\nabla L_a(\mathbf{x}^{(k)})\|^2}$$

- **Polak-Ribière:**

$$\beta_{PR}^{(k)} = \frac{\nabla L_a(\mathbf{x}^{(k+1)})^T \left[\nabla L_a(\mathbf{x}^{(k+1)}) - \nabla L_a(\mathbf{x}^{(k)})\right]}{\|\nabla L_a(\mathbf{x}^{(k)})\|^2}$$

O cálculo do gradiente da função Lagrangeana é realizado dinamicamente em cada passo, considerando as soluções da iteração corrente e da anterior. É importante realçar que a rotina de derivação foi programada para lidar com a natureza não-linear da função, tratando especificamente os operadores "max"presentes na formulação matemática para garantir que o vetor gradiente reflete com precisão a inclinação local.

Para assegurar a convergência e a eficiência da busca, implementou-se um critério de reinício (*restart*) da direção de pesquisa. Esta condição é ativada sempre que a ortogonalidade entre gradientes sucessivos é perdida, conforme expresso pela desigualdade:

$$\left|\nabla L_a(\mathbf{x}^k)^T \nabla L_a(\mathbf{x}^{k-1})\right| > \delta \left\|\nabla L_a(\mathbf{x}^{k-1})\right\|^2 \tag{3.23}$$

Nesta implementação, adotou-se o valor de $\delta = 0.5$. Uma vez estabelecida a direção de pesquisa $\mathbf{d}^k$, o algoritmo determina o comprimento do passo ideal (α^k) para proceder à atualização do vetor de variáveis de projeto:

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{d}^k \tag{3.24}$$

3.4.4 Algoritmo de Pesquisa Linear com *Box Constraints*

Para a determinação do comprimento do passo (α) em cada iteração do Método do Gradiente Conjugado, adotou-se uma variante do algoritmo de pesquisa linear de Armijo. Esta implementação foi especificamente adaptada para integrar as *box constraints*, definidas pelas funções g_5 e g_8 a g_{15}:

$$g_5(\mathbf{x}) = 0.125 - x_1 \leq 0 \tag{3.25}$$

$$\text{Restrições } g_8 \text{ a } g_{11} : 0.1 \leq x_i \leq 2.0, \quad i = 1,4 \tag{3.26}$$

$$\text{Restrições } g_{12} \text{ a } g_{15} : 0.1 \leq x_i \leq 10.0, \quad i = 2,3 \tag{3.27}$$

Notou-se uma sobreposição direta entre as condições impostas por g_5 e g_8. Dado que o limite de 0.125 é mais restritivo que 0.1, a restrição g_8 foi considerada redundante no seu limite inferior, fixando-se 0.125 como o valor mínimo admissível para x_1.

A estratégia de atualização do passo garante que, em cada ciclo, a viabilidade da solução é verificada. Caso as restrições sejam violadas ou o critério de decréscimo insuficiente não seja atingido, o valor de α é iterativamente reduzido:

Passo 1: Determinar o valor inicial Δ^k através da relação:

$$\Delta^k \geq c \frac{|\nabla L_a(\mathbf{x}^k)^T \mathbf{d}^k|}{\|\mathbf{d}^k\|^2} \tag{3.28}$$

e atribuir $\alpha = \Delta^k$.

Passo 2: Enquanto a condição de Armijo não for validada:

$$L_a(\mathbf{x}^k + \alpha \mathbf{d}^k) > L_a(\mathbf{x}^k) + \gamma \alpha \nabla L_a(\mathbf{x}^k)^T \mathbf{d}^k \tag{3.29}$$

$$\alpha \leftarrow \delta \alpha \tag{3.30}$$

Para esta implementação, utilizaram-se os parâmetros de controlo $c = 0.1$, $\gamma = 0.1$ e $\delta = 0.5$.

Por fim, o vetor $\mathbf{x}$ é atualizado por meio da Equação 3.24, sendo posteriormente verificado o cumprimento das *box constraints*. Caso alguma destas restrições não seja satisfeita, os limites admissíveis são imediatamente impostos ao vetor atualizado.

A convergência do processo é então avaliada recorrendo a dois critérios: a norma da diferença entre soluções consecutivas e a variação absoluta no valor da função Lagrangeana. Ambos os indicadores são comparados com a tolerância previamente definida para o método do Gradiente Conjugado.

3.4.5 Estrutura e Implementação Computacional do Algoritmo

A implementação computacional foi desenvolvida em ambiente MATLAB segundo uma arquitetura modular que separa de forma clara as diferentes componentes do método. Esta organização permite distinguir o ciclo externo do Método do Lagrangiano Aumentado, o ciclo interno associado ao Método do Gradiente Conjugado, bem como os módulos responsáveis pelo cálculo numérico dos gradientes e pela avaliação das funções e restrições.

Organização dos Módulos e Funções

A rotina de cálculo está dividida em três camadas principais:

1. **Gestão do Ciclo Externo:** Este módulo coordena o processo global de otimização, atualizando os multiplicadores de Lagrange, λ, e o parâmetro de penalização, ε. Monitoriza a evolução iterativa da solução, avalia critérios de convergência e aciona a verificação formal das condições de otimalidade sempre que necessário.

2. **Minimização do Subproblema:** O ciclo interno de otimização resolve o subproblema associado a valores fixos de λ e ε. A direção de descida é determinada através do método dos gradientes conjugados, utilizando as formulações de Polak-Ribière ou de Fletcher-Reeves para o cálculo do fator de conjugação β. A pesquisa linear segue a condição de *Armijo*, garantindo que o passo α cumpre o critério de decréscimo suficiente e assegura um avanço estável do processo iterativo.

3. **Cálculo Numérico de Gradientes:** O gradiente da função Lagrangiana aumentada foi obtido diretamente recorrendo a esquemas de diferenças finitas, incorporando simultaneamente a contribuição da função objetivo e das restrições. Esta abordagem evitou a necessidade de derivação analítica, assegurando uma implementação robusta e suficientemente precisa para o processo de otimização.

Critérios de Convergência e Verificação KKT

Para garantir que a solução encontrada representa um mínimo local válido, implementou-se uma função de verificação das condições de Karush-Kuhn-Tucker. Após cada ciclo do Lagrangeano Aumentado, o algoritmo valida:

1. **Estacionaridade:** A norma do gradiente da função Lagrangeana ($\|\nabla f + \sum \lambda_i \nabla g_i\|$) deve ser inferior à tolerância de 10^{-4}.

2. **Folga Complementar:** Verifica-se se o produto dos multiplicadores pelas restrições ($\lambda_i \cdot g_i$) é aproximadamente zero.

3. **Admissibilidade Primal:** Garante-se que nenhuma restrição é violada significativamente através da monitorização de $\max(g_i)$.

Gestão de Variáveis e *Box Constraints*

A gestão das variáveis de projeto e das respetivas *box constraints* é integrada diretamente na formulação do problema através da função Lagrangiana Aumentada, definida por:

$$L_a(\mathbf{x}, \lambda, \varepsilon) = f(\mathbf{x}) + \lambda^T \max\left(g(\mathbf{x}), -\frac{\varepsilon}{2}\lambda\right) + \frac{1}{\varepsilon}\left\|\max\left(g(\mathbf{x}), -\frac{\varepsilon}{2}\lambda\right)\right\|^2. \tag{3.31}$$

As restrições do tipo *box* impostas às variáveis de projeto (h, l, t, b) são asseguradas através de uma projeção direta no intervalo admissível ao final de cada iteração do método do gradiente conjugado. A operação é implementada como

$$\mathbf{x} \leftarrow \min\big(\max(\mathbf{x}, \mathbf{lb}), \mathbf{ub}\big),$$

com

$$\mathbf{lb} = [0.125,\ 0.1,\ 0.1,\ 0.1], \qquad \mathbf{ub} = [2.0,\ 10.0,\ 10.0,\ 2.0].$$

Este procedimento garante que todas as variáveis permanecem dentro dos limites físicos estipulados ao longo de todo o processo de otimização.

3.5 Discussão de Resultados

3.5.1 Resultados Comparativos: Fletcher-Reeves vs Polak-Ribière

Nesta secção comparam-se os desempenhos das duas variantes do Método do Gradiente Conjugado utilizadas no ciclo interno do Lagrangeano Aumentado: Polak-Ribière e Fletcher-Reeves. As Tabelas 3.1 e 3.2 apresentam a evolução das variáveis de projeto e do valor da função objetivo em cada iteração do ciclo externo para ambas as formulações.

É de notar que em ambas as análises os resultados obtidos não apresentaram violações nas restrições.

Tabela 3.1: Resultados do Método do Lagrangeano Aumentado para Polak-Ribière (β_{PR})

Iteração	**h**	**l**	**t**	**b**	$f(\mathbf{x})$
0	0.4000	6.0000	9.0000	0.5000	5.3904
1	0.5496	1.5104	9.8814	0.4939	4.600
2	0.2224	6.2051	9.9895	0.2332	2.280
3	0.2314	5.9574	9.4236	0.2374	2.417
4	0.2431	5.8865	8.6497	0.2422	2.399
5	0.2443	6.2026	8.2961	0.2442	2.381
6	0.2437	6.3063	8.2610	0.2448	2.379
7	0.2441	6.2351	8.2920	0.2443	2.381
8	0.2444	6.2136	8.2917	0.2444	2.381
9	0.2444	6.2164	8.2916	0.2444	2.381
10	0.2444	6.2177	8.2914	0.2444	2.381
11	0.2444	6.2177	8.2914	0.2444	2.381

Tabela 3.2: Resultados do Método do Lagrangeano Aumentado para Fletcher-Reeves (β_{FR})

Iteração	**h**	**l**	**t**	**b**	$f(\mathbf{x})$
0	0.4000	6.0000	9.0000	0.5000	5.3904
1	0.2483	5.6726	8.2618	0.2430	2.328
2	0.2456	6.1199	8.3039	0.2444	2.380
3	0.2443	6.2202	8.2913	0.2443	2.381
4	0.2444	6.2173	8.2915	0.2444	2.381
5	0.2444	6.2177	8.2915	0.2444	2.381
6	0.2444	6.2177	8.2915	0.2444	2.381

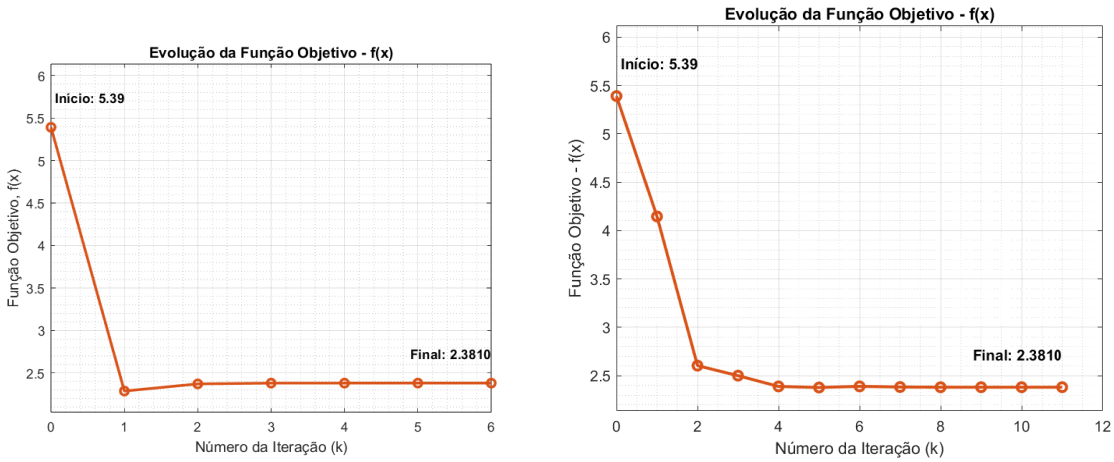
A análise numérica mostra que ambas as abordagens convergem para os mesmos valores finais das variáveis de projeto. Contudo, a velocidade de convergência difere: Fletcher-Reeves alcança o ótimo em 6 iterações, enquanto Polak-Ribière necessita de 11 iterações. Esta diferença é compatível com o comportamento conhecido das duas variantes, sendo a de Polak-Ribière mais sensível a flutuações locais do gradiente.

O ponto ótimo obtido numericamente é:

$$h = 0.2444, \quad l = 6.2177, \quad t = 8.2914, \quad b = 0.2444,$$

com um valor final da função objetivo de $f = 2.381$, em concordância com o valor de referência do problema.

As Figuras 3.1a e 3.1b ilustram a evolução da função objetivo ao longo das iterações. Em ambas as variantes observa-se uma redução acentuada do valor de $f(\mathbf{x})$ nas iterações iniciais, seguida de uma fase de estabilização até à convergência.

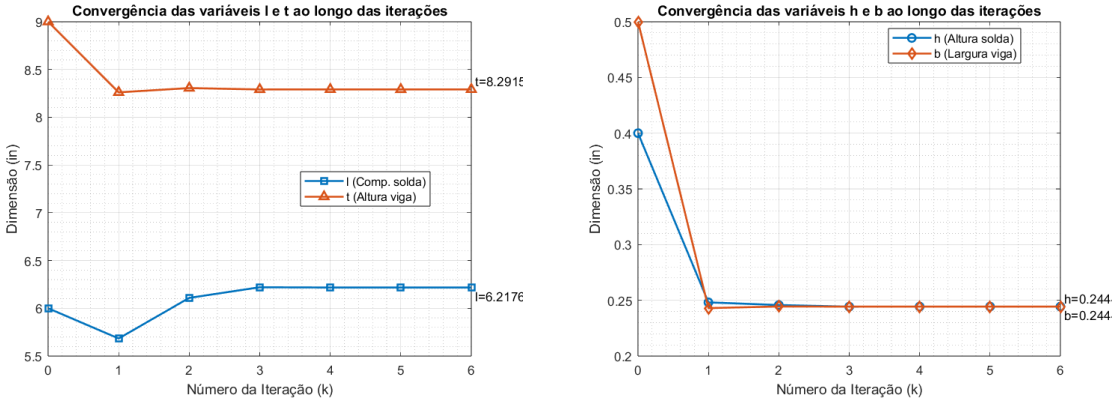


(a) Fletcher-Reeves (b) Polak-Ribière

Figura 3.1: Evolução da função objetivo $f(x)$ para as duas variantes do Método do Gradiente Conjugado.

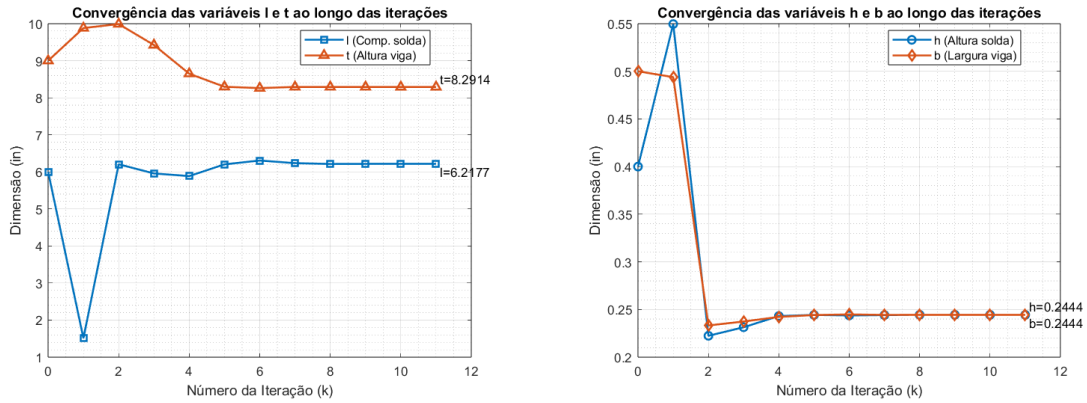
De forma complementar, as Figuras 3.2 e 3.3 representam a evolução das variáveis de projeto. Estas curvas mostram trajetórias típicas de problemas de otimização não linear: uma fase inicial

de ajustes significativos, seguida de aproximação suave ao ponto ótimo.



(a) Evolução de l e t (b) Evolução de h e b

Figura 3.2: Evolução das variáveis de projeto para β_{FR}



(a) Evolução de l e t (b) Evolução de h e b

Figura 3.3: Evolução das variáveis de projeto para β_{PR}

A análise numérica e gráfica permite concluir que ambos os métodos convergem para o mesmo ponto ótimo, o que confirma a consistência da formulação adotada e a fiabilidade do cálculo numérico do gradiente da Lagrangiana Aumentada. Observou-se, contudo, uma diferença clara em termos de eficiência: a variante de Fletcher–Reeves revelou um desempenho mais favorável, alcançando a solução com menos iterações e seguindo uma trajetória mais regular ao longo do processo iterativo. Por sua vez, a formulação de Polak–Ribière apresentou oscilações mais pronunciadas, embora tenha mantido robustez e conduzido igualmente ao ponto ótimo final.

A verificação sistemática das condições de Karush–Kuhn–Tucker confirmou que a solução obtida satisfaz simultaneamente os requisitos de estacionaridade, admissibilidade primal e complementaridade. Assim, apesar das diferenças no comportamento das duas variantes, ambas demonstraram desempenho adequado e convergência robusta, sendo a escolha entre elas predominantemente influenciada pela eficiência pretendida para o problema em estudo.

Capítulo 4

Aplicação de Algoritmos Evolucionários na Resolução do Problema

4.1 Revisão Bibliográfica sobre métodos de pesquisa evolucionária

Os métodos de pesquisa evolucionária constituem uma classe de técnicas de otimização inspiradas nos princípios da evolução natural, amplamente aplicadas na resolução de problemas complexos e não lineares. Estes métodos utilizam populações de soluções e operadores como seleção, recombinação e mutação para explorar o espaço de busca, promovendo um equilíbrio entre diversidade e pressão seletiva. Ao longo das últimas décadas, abordagens como algoritmos genéticos, estratégias evolutivas e programação genética têm sido estudadas e aplicadas em diversos domínios, demonstrando elevada robustez face a problemas com múltiplos objetivos, restrições e superfícies de busca irregulares. Nesta revisão, serão apresentados os conceitos fundamentais, as diferenças entre genótipo e fenótipo, e as principais estratégias utilizadas para garantir eficiência e convergência nos processos evolutivos.

4.1.1 Terminologia Biológica Aplicada aos Algoritmos Genéticos

Os algoritmos genéticos e outros métodos evolucionários são inspirados em processos biológicos, pelo que é comum recorrer a terminologia proveniente da biologia para descrever os seus componentes. Entre os conceitos fundamentais destacam-se: **genótipo**, que representa a codificação interna da solução (por exemplo, uma cadeia binária), e **fenótipo**, que corresponde à interpretação dessa codificação no espaço real do problema. Além disso, termos como *população*, *indivíduo*, *gene*, *cromossoma*, *seleção*, *cruzamento* e *mutação* são utilizados para estabelecer analogias com os mecanismos de evolução natural, facilitando a compreensão das operações realizadas pelo algoritmo.

4.1.2 Estrutura do Algoritmo Genético

Tal como os organismos vivos no seu ambiente natural, os Algoritmos Genéticos (AGs) necessitam de receber informação, $I(\tau)$, sobre a adaptação da população de indivíduos, $P(\tau)$, relativamente às condições ambientais em Ω. Na prática, $I(\tau)$ é fornecida pela função de avaliação dos indivíduos que compõem a população. A partir desta informação, os AGs modificam a população de soluções, alcançando uma adaptação progressivamente melhor ao ambiente em que se encontram inseridos.

De forma geral, pode expressar-se o processo como:

$$\text{AG: } P(\tau) \times I(\tau) \longrightarrow P(\tau+1)$$

O primeiro algoritmo formal, proposto por Holland, é conhecido como *Algoritmo Genético Simples* (SGA). Este constitui a base para a maioria das variantes modernas e caracteriza-se por uma representação binária, seleção baseada na aptidão, baixa probabilidade de mutação e substituição geracional completa. O ciclo evolutivo segue uma lógica iterativa, onde cada geração é construída a partir da anterior através de operadores genéticos. De forma resumida, os passos principais são apresentados no Algoritmo 1 [1].

Algorithm 1: Algoritmo Genético Simples (SGA) — ciclo geracional mínimo.

Data: População inicial $P(0)$, prob. de cruzamento P_c, prob. de mutação por bit P_{mut}, dimensão da população N

Result: População final e melhor indivíduo

START

1. Selecionar os pais para o conjunto de acasalamento (tamanho N);
2. Baralhar/emparelhar o conjunto de acasalamento (mantendo N por geração);
3. Para cada par sucessivo, aplicar o cruzamento com probabilidade P_c; se não cruzar, copiar os pais;
4. Para cada descendente, aplicar mutação: inverter cada bit com probabilidade independente P_{mut};
5. Substituir toda a população pelos descendentes resultantes;

END

Este ciclo é repetido até que seja atingido um critério de paragem, que pode corresponder a um número máximo de gerações, convergência da população ou obtenção de um valor objetivo pré-definido. A simplicidade do SGA torna-o uma referência fundamental para compreender os mecanismos básicos dos algoritmos genéticos, servindo de ponto de partida para variantes mais sofisticadas.

4.1.3 Representação do Espaço de Soluções

Após a definição das variáveis de decisão e do espaço de busca que representa as soluções possíveis, é necessário estabelecer uma forma adequada de representação para os indivíduos da população. Esta representação, designada por *genótipo*, corresponde à codificação interna utili-

zada pelo algoritmo, enquanto o *fenótipo* refere-se à interpretação dessa codificação no espaço real do problema.

De um modo geral, os Algoritmos Genéticos não operam diretamente sobre a representação real das soluções, mas sim sobre uma codificação que permite aplicar os operadores genéticos padrão (seleção, cruzamento, mutação, etc.). Assim, a definição de um formato de codificação é um passo fundamental no desenvolvimento de um AG, pois estabelece a correspondência entre o conjunto de soluções do problema original (espaço fenotípico) e o conjunto de indivíduos codificados (espaço genotípico).

A escolha da representação influencia diretamente a eficácia do algoritmo, uma vez que determina a forma como as soluções são manipuladas e recombinadas. Para cada indivíduo deve existir uma correspondência única entre genótipo e fenótipo, garantindo que a decodificação seja inversa e sem ambiguidades. Entre os formatos mais comuns destacam-se:

- **Codificação binária**: representação das variáveis através de cadeias de *bits*.
- **Codificação inteira**: adequada para problemas com variáveis discretas.
- **Codificação real**: utilizada em problemas contínuos.
- **Codificação baseada em vetores, listas ou matrizes**: para estruturas mais complexas.

Seguidamente, será detalhado o uso da codificação binária, por ser a forma clássica e mais utilizada nos Algoritmos Genéticos tradicionais.

Codificação Binária

A codificação binária é um dos passos fundamentais na formulação de um Algoritmo Genético. Esta técnica consiste em representar as possíveis soluções do problema (fenótipos) através de uma estrutura binária que constitui o material genético de cada indivíduo (genótipo).

Conhecendo as características do problema, é possível definir o tipo de representação mais adequado, de forma a facilitar o tratamento das soluções. Neste trabalho, optou-se pela codificação binária, que emula a representação real, considerando as diferentes dimensões do vetor de variáveis. Assim, cada variável é codificada e o vetor completo é representado por uma cadeia binária (Figura 4.2), conforme os seguintes passos:

1. Vetor das variáveis: $\mathbf{x} = (x_1, x_2, \ldots, x_n)$,
2. Representação binária (*string*): $S = b_1 b_2 \ldots b_n$ onde cada bloco b_i tem l_i *bits*.

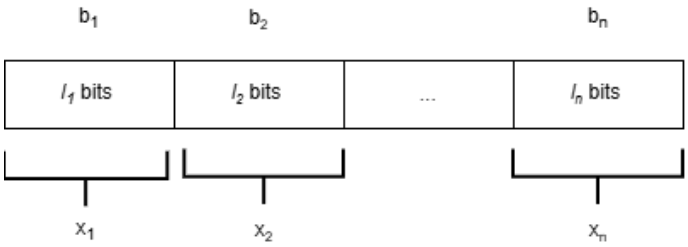


Figura 4.1: Codificação binária: cada variável x_i é representada por um bloco de l_i *bits*.

3. Decodificação da *substring*: $\mathbf{x}_{i,bin}$

$$x_i = x_i^{\inf} + \frac{x_i^{\sup} - x_i^{\inf}}{2^{l_i} - 1} x_{i,\text{bin}} \tag{4.1}$$

com:

$$x_i^{\inf} \leq x_i \leq x_i^{\sup}, \qquad 1 \leq i \leq n \tag{4.2}$$

Para garantir uma precisão de n casas decimais na representação real, considera-se que o menor incremento possível é:

$$\Delta x_i = x_i - x_i^{inf} = 10^{-n} \tag{4.3}$$

A relação entre o valor real e a codificação binária é dada por (4.1). Para o menor incremento, assume-se $x_{i,\text{bin}} = 1$, obtendo:

$$10^{-n} = \frac{x_i^{\sup} - x_i^{\inf}}{2^{l_i} - 1} \tag{4.4}$$

Rearranjando:

$$2^{l_i} - 1 = (x_i^{\sup} - x_i^{\inf}) \cdot 10^n \tag{4.5}$$

Finalmente, o número de *bits* necessário é:

$$l_i = \log_2 \left((x_i^{\sup} - x_i^{\inf}) \cdot 10^n + 1 \right) \tag{4.6}$$

Impacto da precisão na dimensão do espaço de busca:

O número de *bits* l_i necessário para representar cada variável x_i está diretamente relacionado com a precisão desejada, conforme a Equação 4.6. Um aumento no número de casas decimais n implica um incremento em l_i, o que conduz a um maior número de combinações possíveis para essa variável.

A dimensão do domínio de cada variável codificada em binário é dada por:

$$N_i = 2^{l_i} - 1 \tag{4.7}$$

onde N_i representa o número total de soluções discretas possíveis para x_i.

Assim, quanto maior for a precisão requerida (maior n), maior será l_i e, consequentemente, maior será N_i. Este aumento do espaço de busca tem duas implicações principais:

- **Maior diversidade potencial na população:** Um espaço de busca mais amplo permite que os indivíduos iniciais sejam mais variados, reduzindo a probabilidade de convergência prematura.
- **Redução do efeito da elitização:** Com mais soluções possíveis, a influência dos indivíduos mais aptos tende a ser menos dominante nas primeiras gerações, favorecendo a exploração.

Por outro lado, este aumento também implica um maior esforço computacional para explorar o espaço de soluções, podendo tornar o processo mais lento. Em problemas que exigem elevada precisão, uma alternativa frequentemente considerada é a utilização de codificação real, que evita o crescimento exponencial do espaço de busca associado à codificação binária.

4.1.4 Definição do Mérito ou Aptidão

A função de mérito, ou aptidão, é um componente central nos Algoritmos Genéticos, pois determina a qualidade das soluções e orienta o processo evolutivo. A forma como esta função é definida depende do processo de avaliação e pode variar conforme a natureza do problema e os objetivos do algoritmo. Entre as abordagens mais comuns encontram-se: utilizar diretamente o valor da função objetivo, aplicar processos de normalização para tornar os valores comparáveis ou recorrer a critérios relativos, como *rankings*, para ajustar a pressão seletiva e evitar convergência prematura.

Neste trabalho, optou-se por uma formulação baseada na normalização, que assegura que os valores da função de mérito permanecem positivos e comparáveis, mesmo quando a função objetivo assume valores elevados ou quando existem penalizações associadas a restrições. A função de mérito é definida por:

$$\mathrm{Mer}(x_i) = C_{\max} - \left[f(x_i) + \Gamma(x_i)\right] \tag{4.8}$$

onde $f(x_i)$ representa o valor da função objetivo do problema (função positiva a minimizar), $\Gamma(x_i) \geq 0$ é um termo de penalização associado à violação das restrições, e $C_{\max}$ é um escalar escolhido para garantir que a função de mérito seja sempre positiva.

Esta formulação apresenta duas vantagens fundamentais. Em primeiro lugar, permite normalizar os valores de aptidão, facilitando a aplicação dos operadores de seleção. Em segundo lugar, a estrutura da Equação 4.8 possibilita compreender de forma clara a evolução da aptidão ao longo das gerações, uma vez que está diretamente relacionada com a melhoria da função objetivo e com a redução das penalizações. Assim, a função de mérito não só orienta a busca para soluções viáveis e otimizadas, como também fornece um indicador intuitivo da qualidade das soluções durante todo o processo evolutivo.

4.1.5 Métodos de Penalização em Algoritmos Genéticos

Em problemas com restrições, é comum que soluções inviáveis surjam durante o processo evolutivo. Para lidar com esta situação, utilizam-se funções de penalização que reduzem a aptidão das soluções que violam restrições, tornando-as menos prováveis de serem selecionadas. Existem

diferentes estratégias para aplicar penalizações. A penalização estática consiste em atribuir valores fixos, independentemente da geração, o que pode levar à convergência prematura. A penalização dinâmica, por sua vez, ajusta a severidade ao longo das gerações, permitindo maior diversidade inicial e reforçando a viabilidade à medida que o algoritmo evolui. Finalmente, a penalização adaptativa baseia-se no comportamento da população, alterando os parâmetros conforme a evolução, de forma a equilibrar a pressão seletiva com a exploração do espaço de busca.

Entre as abordagens mais referenciadas na literatura destacam-se os métodos propostos por Joines et al. e Hadj-Alouane & Bean, que serão descritos em seguida.

1. **Joines et al. [4]:** A penalização aumenta com a geração segundo $(Ck)^{\alpha}$, conforme a Equação 4.13, controlando a pressão seletiva com parâmetros fixos.

2. **Hadj-Alouane & Bean [2]:** A penalização é adaptativa, ajustando $\lambda(k)$ dinamicamente com base na viabilidade das melhores soluções, conforme a Equação 4.14.

Estas estratégias permitem equilibrar a exploração do espaço de busca com a satisfação das restrições, evitando convergência prematura e garantindo soluções viáveis.

4.1.6 Operadores Genéticos de Seleção

A seleção dos indivíduos para o cruzamento foi realizada através do método *Stochastic Universal Sampling* (SUS), uma variação do método da roleta (*roulette wheel selection*). Neste procedimento, cada indivíduo recebe uma probabilidade proporcional ao seu mérito, calculada por:

$$P_i = \frac{\text{Mer}(x_i)}{\sum_{j=1}^{N_{\text{pop}}} \text{Mer}(x_j)} \tag{4.9}$$

onde $\text{Mer}(x_i)$ é o mérito do indivíduo i e N_{pop} o tamanho da população.

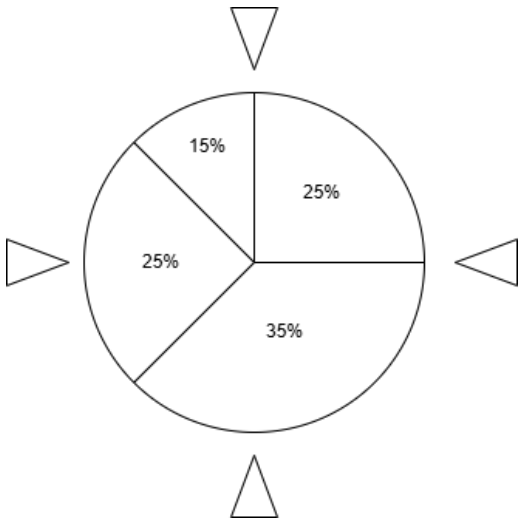


Figura 4.2: Exemplo de Seleção através do Método *Stochastic Universal Sampling* (SUS).

O SUS utiliza marcadores igualmente espaçados sobre a distribuição acumulada das probabilidades, garantindo que todos os indivíduos têm uma oportunidade proporcional de serem selecionados. Este método reduz a aleatoriedade excessiva e assegura uma pressão seletiva equilibrada, preservando a diversidade da população.

4.1.7 Operadores Genéticos de Recombinação e Cruzamento

O processo de recombinação tem como objetivo gerar novos cromossomas a partir de indivíduos selecionados, transferindo material genético dos pais para os descendentes. Este mecanismo está associado aos conceitos de hereditariedade, que garante a transmissão das características dos progenitores, e de validade, assegurando que as soluções geradas permanecem admissíveis no espaço de busca.

Segundo a taxonomia proposta por Herrera et al. [3], os operadores de cruzamento podem ser agrupados em três categorias principais:

- ***Discrete Crossover Operators* (DCOs):** Cada gene do descendente é obtido diretamente de um dos pais, sem transformação numérica. Exemplos incluem o cruzamento de um ponto, dois pontos e o cruzamento uniforme.
- ***Aggregation-Based Crossover Operators* (ABCOs):** Combinam numericamente os valores dos genes dos pais, utilizando funções de agregação.
- ***Neighbourhood-Based Crossover Operators* (NBCOs):** Geram genes a partir de intervalos definidos em torno dos valores dos pais, recorrendo a funções de probabilidade.

Neste trabalho, foi utilizado o *Uniform Crossover*, pertencente à categoria DCO, por ser adequado à codificação binária e por promover uma elevada diversidade genética. Neste operador, cada gene do descendente é escolhido aleatoriamente de um dos pais com probabilidade uniforme, garantindo uma mistura equilibrada do material genético e preservando a validade das soluções.

Cruzamento Uniforme

O cruzamento uniforme é um operador de recombinação que promove a troca equilibrada de material genético entre dois progenitores. Diferentemente dos métodos de ponto único ou múltiplos pontos, este operador considera cada posição do cromossoma individualmente, garantindo uma mistura uniforme dos genes dos pais. Para controlar esta troca, utiliza-se um vetor máscara $\mathbf{M}$, composto por valores binários $\{0,1\}$, cuja dimensão l_c corresponde ao comprimento dos cromossomas. Cada componente M_i indica a origem do gene na posição i: quando $M_i = 0$, o gene do primeiro descendente é herdado do primeiro progenitor e o segundo descendente do segundo progenitor; quando $M_i = 1$, a atribuição é invertida. Este mecanismo assegura diversidade genética, preserva a validade da codificação binária e garante que todos os genes dos descendentes provêm dos pais (hereditariedade).

O procedimento básico para aplicar este operador é apresentado no Algoritmo 2, onde se observa a utilização da máscara para definir a origem dos genes em cada posição:

Algorithm 2: Cruzamento uniforme utilizando máscara binária

Data: Pais $P_1, P_2 \in \{0,1\}^{l_c}$; comprimento l_c
Result: Descendentes $S_1, S_2 \in \{0,1\}^{l_c}$
Gerar máscara $\mathbf{M} \in \{0,1\}^{l_c}$ com $M_i \sim \text{Bernoulli}(0.5)$;
for $i \leftarrow 1$ **to** l_c **do**
  if $M_i = 0$ **then**
    $S_1(i) \leftarrow P_1(i); \quad S_2(i) \leftarrow P_2(i)$
  else
    $S_1(i) \leftarrow P_2(i); \quad S_2(i) \leftarrow P_1(i)$

A Figura 4.3 ilustra este processo, mostrando a máscara binária **M**, os cromossomas dos pais (P_1 e P_2) e os descendentes (S_1 e S_2) gerados pelo cruzamento uniforme. Cada célula representa um gene, com cores distintas para indicar a origem do material genético.

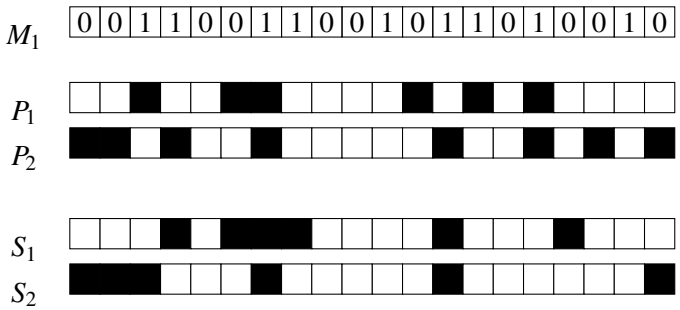


Figura 4.3: Representação de Sequências Binárias e Padrões Correspondentes.

4.1.8 Operadores de Mutação

A mutação é um operador unário que atua sobre um único indivíduo, introduzindo pequenas alterações no seu genótipo com o objetivo de preservar a diversidade da população e evitar convergência prematura. A intensidade da mutação é controlada pela probabilidade P_m, que define a chance de cada gene ser alterado. Valores típicos para P_m situam-se no intervalo $[0.001, 0.01]$, podendo também ser inicializados como $1/k$, onde k é o número de variáveis de decisão, garantindo que, em média, apenas um gene é modificado por indivíduo.

Para que um operador de mutação seja eficaz, deve respeitar três propriedades fundamentais:

- **Ergodicidade:** permitir que todas as soluções do espaço de busca possam ser alcançadas.
- **Validade:** produzir soluções admissíveis, respeitando as restrições do problema.
- **Localidade:** assegurar que pequenas alterações no genótipo resultam em pequenas alterações no fenótipo, evitando perturbações excessivas.

Mutação por Troca de Genes (*Flip-Bit*)

No caso da codificação binária, foi utilizada a mutação do tipo *flip-bit*, que consiste em inverter o valor de um gene selecionado aleatoriamente (0 torna-se 1 e vice-versa) com uma probabilidade P_m. Este operador é simples, eficiente e garante a preservação da validade da representação binária, mantendo a diversidade genética na população. A Figura 4.4 ilustra este processo.

Algorithm 3: Mutação *flip-bit* para codificação binária

Data: Indivíduo $I \in \{0,1\}^{l_c}$; probabilidade de mutação P_m
Result: Indivíduo mutado I'
for $i \leftarrow 1$ **to** l_c **do**
  if $rand() < P_m$ **then**
    if $I(i) = 0$ **then**
      $I'(i) \leftarrow 1$
    else
      $I'(i) \leftarrow 0$
  else
    $I'(i) \leftarrow I(i)$

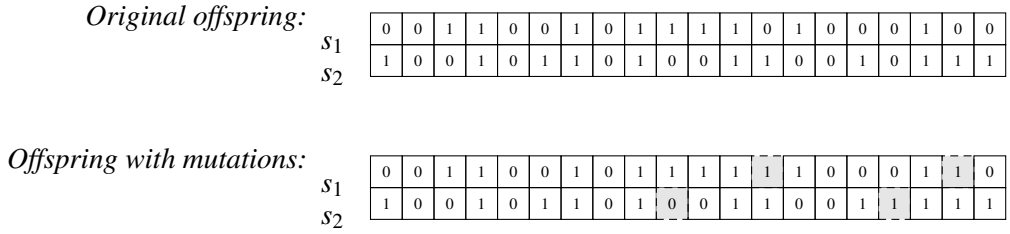


Figura 4.4: O Operador de Mutação *flip-bit*.

4.2 Implementação do Algoritmo Genético

A implementação do algoritmo genético foi desenvolvida com o objetivo de traduzir os conceitos teóricos apresentados anteriormente para uma solução computacional eficiente e adaptada ao problema em estudo. Nesta secção descrevem-se as principais decisões de modelação, a estrutura do código e os mecanismos utilizados para representar indivíduos, avaliar a função de mérito, aplicar operadores genéticos (seleção, cruzamento e mutação) e gerir a evolução da população ao longo das gerações.

Para garantir a diversidade e a convergência do processo evolutivo, é essencial definir um conjunto de parâmetros fundamentais que controlam o comportamento do algoritmo. A Tabela 4.1 apresenta os valores adotados para este trabalho, incluindo o tamanho da população, o número de indivíduos de elite, as probabilidades associadas aos operadores genéticos e o número máximo de gerações. Estes parâmetros foram escolhidos com base em boas práticas da literatura e ajustados de forma a equilibrar a exploração e a intensificação da busca.

Tabela 4.1: Parâmetros Fundamentais do Algoritmo Genético

Parâmetro	**Valor**
Tamanho da população (N_{pop})	30
Percentagem da elite (P_{elite})	0.3
Probabilidade de cruzamento (P_{cross})	0.9
Probabilidade de mutação (P_{mut})	0.008
Número máximo de gerações	100

4.2.1 Definição do mérito

A avaliação dos indivíduos é baseada na função de mérito, definida como:

$$Mer(x) = C_{\max} - Aval(x), \tag{4.10}$$

onde $Aval(x)$ representa o valor da função do método da penalidade dinâmica e $C_{\max}$ é um limite superior fixado para garantir que o mérito seja sempre não negativo. Neste trabalho, definiu-se $C_{\max} = 100$, permitindo acomodar uma maior diversidade de indivíduos com diferentes níveis de penalização.

Para evitar que indivíduos com valores de avaliação excessivamente elevados assumam um papel desproporcional na seleção, aplica-se uma saturação:

$$Aval(x) = \min\left(Aval(x), C_{\max} - 10^{-6}\right), \tag{4.11}$$

garantindo que mesmo indivíduos penalizados participem no processo evolutivo, embora com um impacto reduzido.

A função $Aval(x)$ segue o método da penalidade dinâmica, que permite incorporar restrições do tipo desigualdade no processo evolutivo. Como o problema em estudo não inclui restrições de igualdade ($h(x) = 0$), mas apenas restrições de desigualdade, a violação é considerada apenas quando a restrição é ultrapassada. Para isso, define-se:

$$u_j(x) = \max\left[0, g_j(x)\right], \tag{4.12}$$

onde $g_j(x)$ representa a restrição j. Assim, se $g_j(x) \leq 0$, a restrição está satisfeita e $u_j(x) = 0$; caso contrário, $u_j(x)$ assume o valor positivo da violação. As restrições normalizadas foram apresentadas no Capítulo 3.4.1.

Foram implementadas duas formas distintas de calcular $Aval(x)$ no código, com o objetivo de, no final, comparar os seus efeitos no desempenho do algoritmo e na qualidade das soluções obtidas:

O método da penalidade dinâmica proposto por Joines et al. [4] defende que, no caso de um

problema de minimização, as soluções devem ser avaliadas por geração k através da expressão:

$$Aval(x) = f(x) + (Ck)^{\alpha} \sum_{j=1}^{J} [u_j(x)]^{\beta} \tag{4.13}$$

onde $f(x)$ é a função objetivo, C é uma constante, e α e β são parâmetros que controlam a severidade da penalização ao longo das gerações.

Os valores das constantes utilizados neste trabalho são apresentados na Tabela 4.2.

Tabela 4.2: Constantes Utilizadas na Função de Penalização Dinâmica (Método de Joines et al.)

Parâmetro	**Descrição e Valor**
C	Constante base para penalização: **0.5**
α	Expoente para evolução da penalização: **2**
β	Expoente aplicado às violações das restrições: **2**

Um processo alternativo, utilizado neste trabalho, foi o método proposto por Hadj-Alouane & Bean [2], no qual a penalidade depende do processo evolutivo, sendo dada por:

$$Aval(x) = f(x) + \lambda(k) \sum_{j=1}^{J} [u_j(x)]^2 \tag{4.14}$$

onde $\lambda(k)$ é atualizado dinamicamente ao longo das gerações, permitindo ajustar a severidade da penalização conforme o processo evolutivo avança, como pode ser observado no Algoritmo 4.

Algorithm 4: Atualização de $\lambda(k)$ baseada no método proposto por **Hadj-Alouane and Bean [2]**

Data: $\beta_1 > \beta_2 > 1$, valor inicial λ_1, N_f (frequência de alteração), população inicial
Result: Sequência $\lambda(k)$ atualizada ao longo das gerações
Inicialização:
Gerar aleatoriamente a população e avaliar $Aval(x)$; definir $k \leftarrow 1$.
Passo Principal:
while *(não parar)* **do**
  Criar uma nova geração com o algoritmo genético e avaliar objetivos/penalizações;
  if *as **últimas** N_f gerações tiveram **topo** com penalização* ($\sum_j u_j(x_{best}) > 0$) **then**
    definir $\lambda_{k+1} \leftarrow \beta_1 \lambda_k$; reavaliar a geração com λ_{k+1};
  else if *as **últimas** N_f gerações tiveram **topo** sem penalização* ($\sum_j u_j(x_{best}) = 0$) **then**
    definir $\lambda_{k+1} \leftarrow \lambda_k / \beta_2$; reavaliar a geração com λ_{k+1};
  else $\lambda_{k+1} \leftarrow \lambda_k$;
  ;
  $k \leftarrow k+1$;
  if *atingir valor-alvo ou nº máximo de gerações* **then**
    parar

Após a definição do algoritmo de atualização dinâmica de $\lambda(k)$, foram considerados três ce-

nários distintos para avaliar o impacto dos parâmetros iniciais na convergência e qualidade das soluções. Em todos os casos, manteve-se $\beta_1 = 1.3$, $\beta_2 = 1.1$ e $N_f = 20$, variando apenas o valor inicial de λ.

É importante salientar que, em ambas as formulações apresentadas nas Equações 4.13 e 4.14, o propósito central é penalizar soluções que violem restrições, garantindo que estas tenham menor probabilidade de serem selecionadas. Ao mesmo tempo, procura-se manter diversidade suficiente na população para evitar convergência prematura, equilibrando a pressão seletiva com a capacidade de explorar novas regiões do espaço de busca.

4.2.2 Estratégias Elitistas

Para melhorar o desempenho do algoritmo genético, é fundamental garantir que os melhores indivíduos da população participem no processo de reprodução. Sem esta salvaguarda, soluções de elevada qualidade podem ser perdidas devido à aplicação dos operadores de cruzamento ou mutação. A estratégia elitista consiste precisamente em preservar uma fração dos indivíduos mais aptos, assegurando que estes são mantidos na população seguinte, independentemente das operações evolutivas.

No presente trabalho, implementou-se uma abordagem elitista baseada no mérito, reservando 30% da população total para os indivíduos com melhor avaliação. Esta percentagem foi escolhida de forma a equilibrar a pressão seletiva com a manutenção da diversidade genética, evitando a convergência prematura para soluções subótimas. Embora a elitização aumente a probabilidade de evolução consistente, é essencial complementá-la com mecanismos que promovam a exploração do espaço de busca, garantindo que novas regiões são investigadas e que a população não se torna excessivamente homogénea.

4.2.3 Critérios de Paragem

A definição de critérios de paragem é essencial para garantir que o algoritmo genético não executa indefinidamente e que o processo evolutivo termina de forma controlada. De forma geral, os critérios mais comuns incluem:

1. Limite máximo de gerações;
2. Limite máximo de avaliações da função objetivo;
3. Ausência de melhoria significativa na qualidade das soluções ao longo de um número definido de gerações.

No presente trabalho, foram utilizadas duas condições principais para determinar o fim do processo evolutivo. A primeira consiste na definição de um número máximo de gerações. Após testes preliminares com um limite de 800 gerações, verificou-se que este valor era excessivo e não trazia melhorias significativas na qualidade das soluções. Assim, optou-se por reduzir progressivamente

este limite até se atingir um valor adequado. O número máximo de gerações foi fixado em 100, garantindo um bom equilíbrio entre tempo computacional e qualidade das soluções obtidas.

A segunda condição está relacionada com a estagnação do melhor indivíduo. Caso o melhor indivíduo da população não apresente melhoria dentro de uma tolerância definida durante 25 gerações consecutivas, o algoritmo é interrompido. Este critério permite evitar desperdício de recursos computacionais quando o processo evolutivo atinge um estado de convergência.

Estas condições foram escolhidas de forma a assegurar que o algoritmo termina quando a probabilidade de encontrar soluções significativamente melhores se torna reduzida, mantendo a eficiência do processo.

4.3 Resultados

Nesta secção apresentam-se os resultados obtidos após a execução do algoritmo genético sob diferentes condições, com o objetivo de avaliar o impacto dos parâmetros na qualidade das soluções. Serão apresentados os valores de mérito, da função objetivo $f(x)$ e das variáveis correspondentes às melhores execuções (*best runs*) para cada cenário considerado.

As simulações foram realizadas com um número máximo de gerações fixado em 100, frequência de atualização $N_f = 20$ e três valores distintos para o parâmetro inicial λ: 10, 20 e 25. Estes cenários permitem analisar a influência da severidade inicial da penalização no processo evolutivo e na convergência das soluções. Para garantir a reprodutibilidade, foi utilizada uma *seed* base constante, sendo que para cada cenário foram efetuadas 25 execuções independentes com sementes controladas.

Os restantes parâmetros do algoritmo, incluindo tamanhos populacionais, estratégias de elitismo e constantes da função de penalização, foram previamente definidos e detalhados na Secção 4.2. Importa ainda referir que foi utilizada uma precisão de 3 casas decimais na representação das variáveis, resultando em comprimentos de genótipos de 11, 14, 14 e 11 *bits* para as quatro variáveis, perfazendo um total de 50 *bits* por indivíduo.

4.3.1 Resultados por cenário de λ

Nesta subsecção apresentam-se os melhores resultados (*best runs*) para os três valores de λ considerados, incluindo o mérito, o valor da função objetivo $f(x)$, o número de restrições violadas, a violação máxima observada e a geração de paragem.

Tabela 4.3: Resumo dos Melhores Resultados por Cenário de λ.

λ	Run	Mérito	$f(x)$	Restr. violadas	Max. violação	k_{stop}
10	06	97.594882	2.387068	3	0.012344	100
20	25	97.615101	2.380363	2	0.005381	100
25	01	97.590343	2.405199	1	0.007902	100

Em todos os cenários, o motivo de paragem foi *max_gen atingido* na geração $k = 100$, evi-

denciando a estabilidade do processo face ao critério de paragem por limite máximo de gerações. Como se observa na Tabela 4.3, os melhores indivíduos apresentam pequenas violações das restrições, com diferenças na severidade conforme o valor de λ. De forma geral, $\lambda = 20$ revelou o melhor compromisso entre função objetivo e penalização, enquanto $\lambda = 10$ apresentou maior violação e $\lambda = 25$ menor número de restrições violadas, mas com $f(x)$ mais elevado. Estes resultados sugerem que uma severidade inicial intermédia favorece soluções com melhor *trade-off* entre desempenho e conformidade.

Os melhores indivíduos (em representação real) para cada cenário foram:

- $\lambda = 10$ (Run 06): $x_{\text{best}} = [0.240413,\ 5.767588,\ 8.839169,\ 0.240156]$;
- $\lambda = 20$ (Run 25): $x_{\text{best}} = [0.241329,\ 6.287878,\ 8.300751,\ 0.243869]$;
- $\lambda = 25$ (Run 01): $x_{\text{best}} = [0.241329,\ 6.326552,\ 8.222194,\ 0.248510]$.

Apesar das diferenças nos vetores solução, os resultados globais foram bastante próximos, indicando que a penalização dinâmica assegurou pressão seletiva suficiente para direcionar a população para regiões promissoras do espaço de busca, mesmo com pequenas violações das restrições.

Entre os três cenários, $\lambda = 20$ destacou-se por apresentar o melhor equilíbrio entre qualidade da solução e conformidade com as restrições. O caso de $\lambda = 10$ obteve um desempenho competitivo, mas com maior severidade nas violações, enquanto $\lambda = 25$ reduziu o número de restrições violadas, embora à custa de um valor objetivo mais elevado. Estes resultados sugerem que uma severidade inicial intermédia ($\lambda = 20$) favorece soluções com melhor *trade-off* entre desempenho e viabilidade.

Análise da Melhor Execução

Para compreender em detalhe o comportamento do algoritmo, analisou-se a melhor execução obtida no cenário com $\lambda = 20$ (*Run* 25), que apresentou o maior mérito e o menor valor da função objetivo entre os casos estudados.

A título de referência, a solução ótima teórica fornecida pelo enunciado é:

$$x^* = \{0.2444,\ 6.2177,\ 8.2915,\ 0.2444\}, \quad f^* = 2.3810,$$

Verifica-se que a solução obtida pelo algoritmo é praticamente coincidente com a ótima, apresentando pequenas diferenças nas variáveis e uma discrepância de apenas 0.0006 na função objetivo, o que confirma a elevada precisão do método, apesar das reduzidas violações das restrições.

Evolução da Função Objetivo e Mérito: A Figura 4.5 mostra a evolução de $f(x)$ do melhor indivíduo ao longo das gerações, evidenciando uma redução abrupta nas primeiras iterações e estabilização após cerca de 20 gerações. De forma consistente, a Figura 4.6 apresenta o histórico do mérito, onde se observa convergência rápida e manutenção da elite próxima do melhor indivíduo, indicando baixa diversidade após a fase inicial.

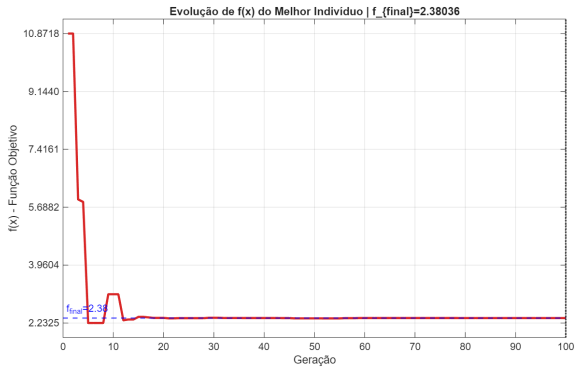


Figura 4.5: Evolução de $f(x)$ do Melhor Indivíduo ao Longo das Gerações (*Run* 25)

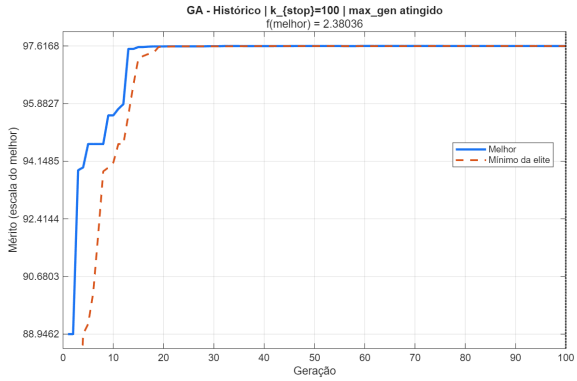


Figura 4.6: Histórico do mérito ao longo das gerações, comparando o melhor indivíduo com o mínimo da elite.

Comportamento das Variáveis e Penalização: A Figura 4.8 ilustra a evolução das quatro variáveis do melhor indivíduo, com ajustes significativos nas primeiras gerações e posterior estabilização próxima dos limites inferiores. A Figura 4.7 evidencia a redução progressiva da penalização e o aumento escalonado de λ, refletindo o mecanismo dinâmico de penalização.

Figura 4.7: Penalização do melhor indivíduo e evolução de λ ao longo das gerações (método dinâmico).

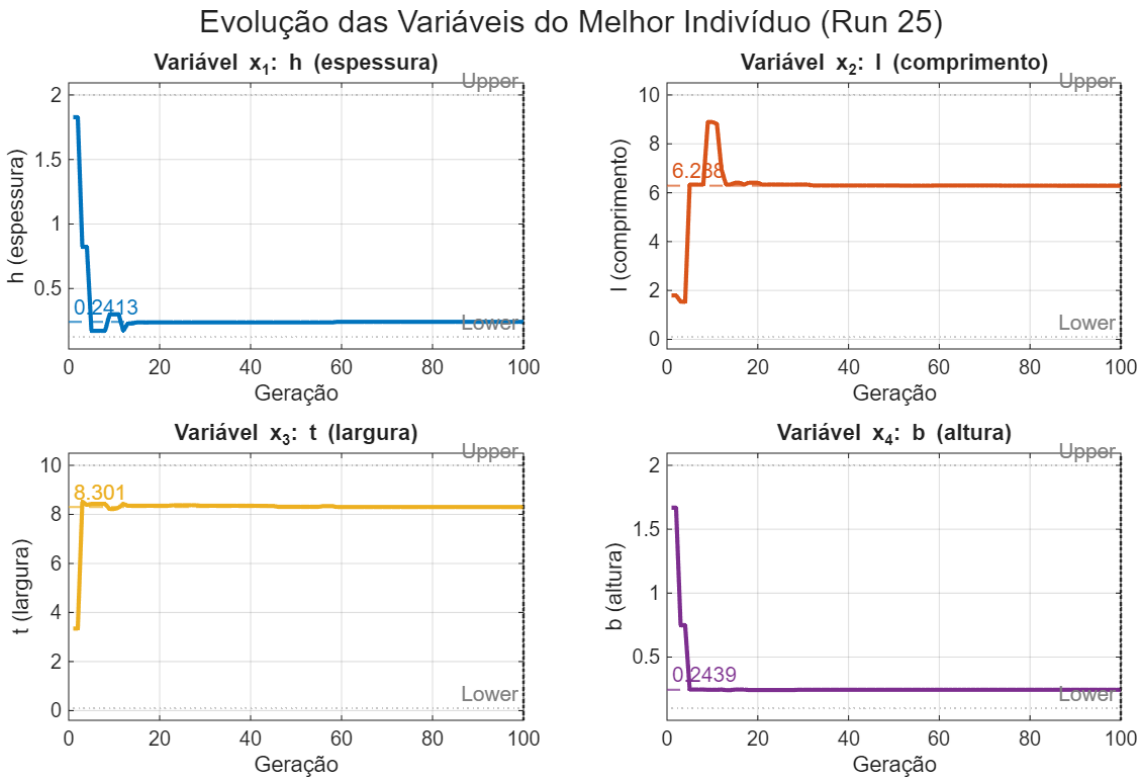


Figura 4.8: Evolução das variáveis do melhor indivíduo (*Run* 25): x_1 (espessura h), x_2 (comprimento l), x_3 (largura t) e x_4 (altura b).

Distribuição da População e Entropia: As Figuras 4.10 mostram a distribuição da população em diferentes gerações, revelando elevada dispersão inicial e concentração progressiva em torno da região ótima, com a elite destacada. Por fim, a Figura 4.9 apresenta a evolução da entropia por variável nos 15 melhores indivíduos, confirmando a perda gradual de diversidade ao longo do processo evolutivo.

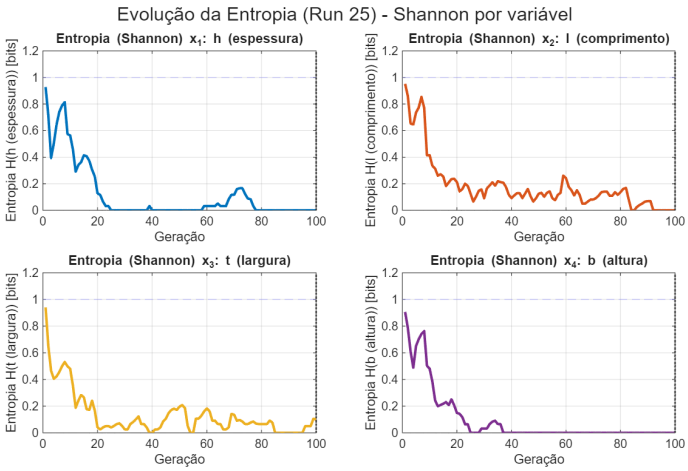
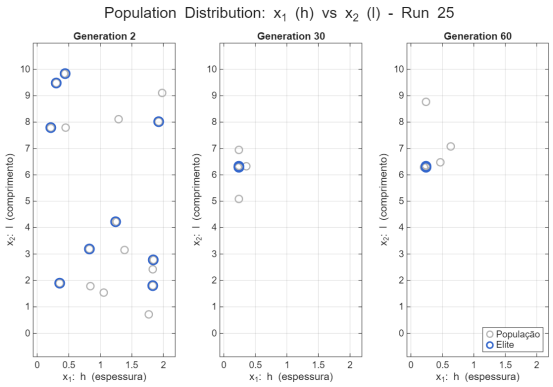
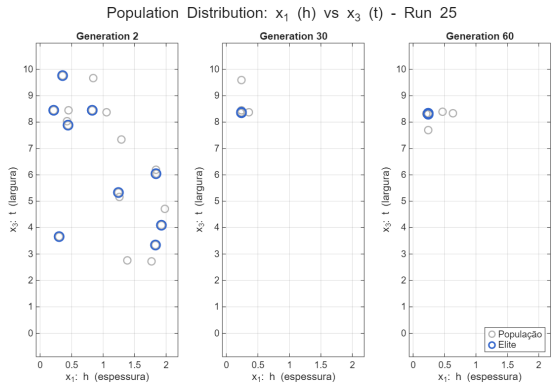


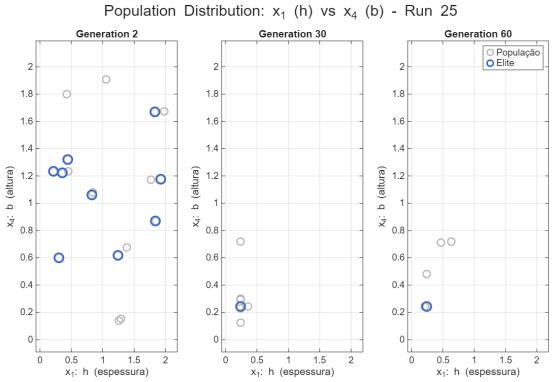
Figura 4.9: Evolução da entropia (Shannon) por variável ao longo das gerações (Run 25), evidenciando perda de diversidade após a convergência.



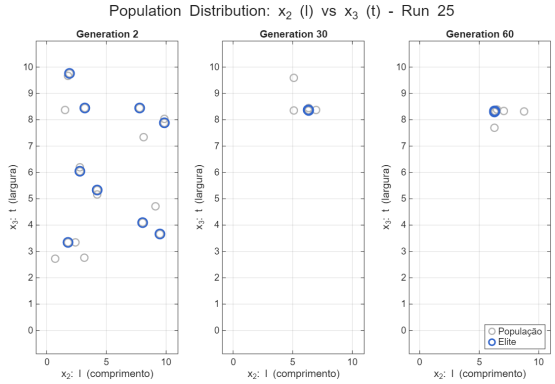
(a) Distribuição: x_1 (h) vs x_2 (l)



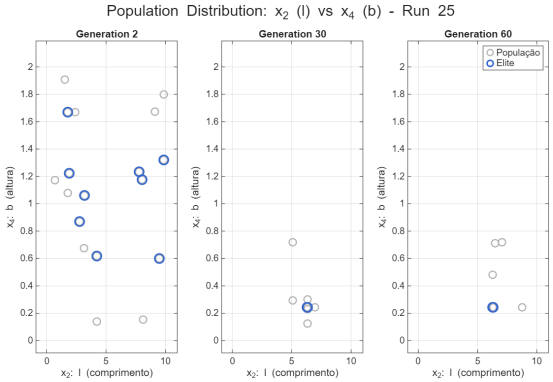
(b) Distribuição: x_1 (h) vs x_3 (t)



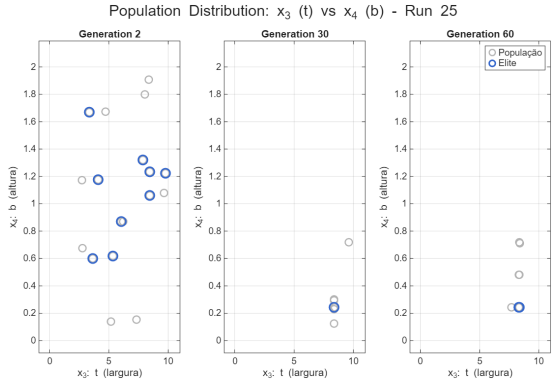
(c) Distribuição: x_1 (h) vs x_4 (b)



(d) Distribuição: x_2 (l) vs x_3 (t)



(e) Distribuição: x_2 (l) vs x_4 (b)



(f) Distribuição: x_3 (t) vs x_4 (b)

Figura 4.10: Distribuição da população e da elite em diferentes gerações (*Run* 25)

Esta análise demonstra que o algoritmo convergiu rapidamente para uma região promissora do espaço de busca, embora com ligeira violação das restrições. A penalização dinâmica mostrou-se eficaz na redução das violações, mas a perda precoce de diversidade sugere que ajustes nos parâmetros ou aumento do número de gerações poderiam melhorar a viabilidade sem comprometer a qualidade da solução.

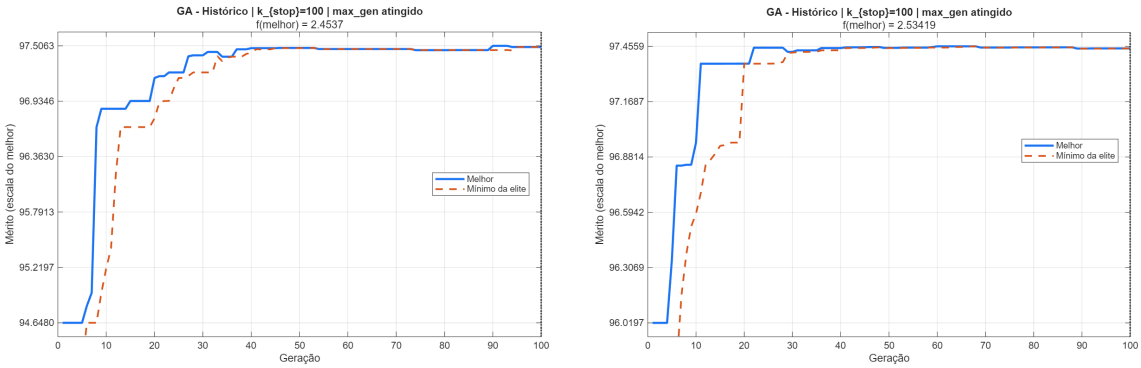
Análise da Influência do número de bits

Observa-se na Tabela 4.4 que a precisão influencia tanto a qualidade da solução como a viabilidade. A configuração com três casas decimais (50 *bits*) apresentou o melhor compromisso entre valor da função objetivo e penalização, refletindo-se no maior mérito. Com duas casas decimais, a granularidade reduzida limitou a exploração, resultando em maior violação das restrições. Por outro lado, aumentar para quatro ou cinco casas decimais não trouxe melhorias significativas; pelo contrário, verificou-se um ligeiro aumento do valor de $f(x)$ e da violação máxima, sugerindo que a maior complexidade do espaço de busca não foi compensada pelo número de gerações fixado (100). Estes resultados indicam que uma precisão intermédia (três casas decimais) é adequada para este problema, equilibrando diversidade e capacidade de convergência.

Tabela 4.4: Impacto da precisão na melhor execução para $\lambda = 20$.

Casas decimais	Bits	Mérito	$f(x)$	Restr. violadas	Max. violação
2	36	97.494290	2.453697	3	0.018866
3	50	97.615101	2.380363	2	0.005381
4	64	97.551887	2.425224	1	0.020018
5	76	97.445165	2.534191	2	0.010072

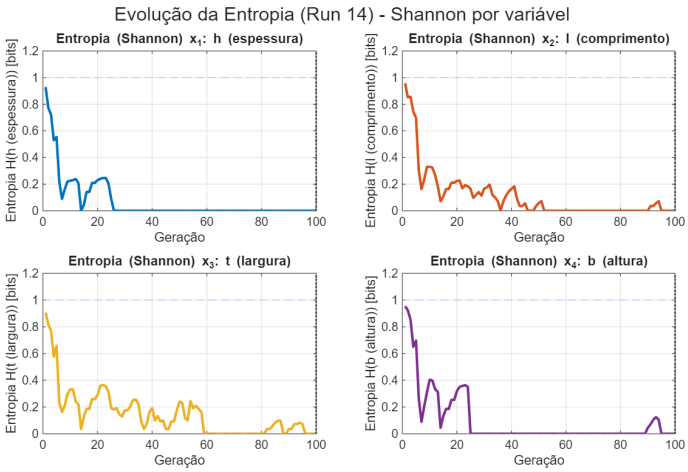
A Figura 4.11 mostra que, para 2 casas decimais, a convergência do mérito ocorre rapidamente, mas com menor valor final, refletindo limitações na granularidade da busca. Já para 5 casas decimais, observa-se maior oscilação inicial e convergência mais lenta, sugerindo maior complexidade do espaço de busca sem ganhos significativos no mérito.



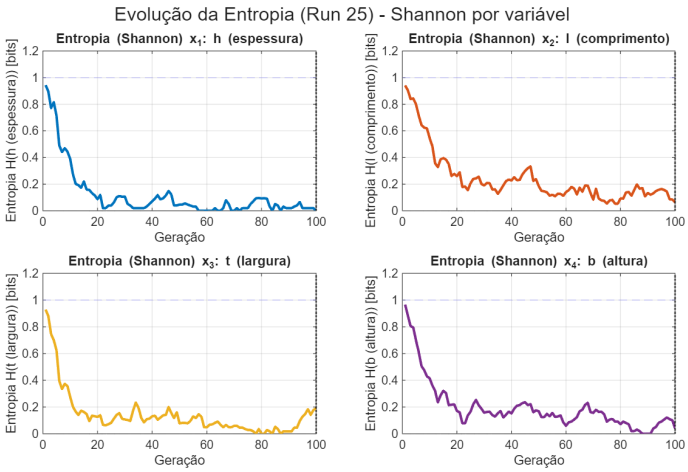
(a) 2 casas decimais (36 *bits*) (b) 5 casas decimais (76 *bits*)

Figura 4.11: Comparação da evolução do mérito do melhor indivíduo e da elite para diferentes precisões (2 vs 5 casas decimais).

A Figura 4.12 evidencia que a entropia decresce mais rapidamente no caso de 2 casas decimais, indicando perda precoce de diversidade. Para 5 casas decimais, a redução é mais gradual, mas não impede a convergência, reforçando que maior precisão aumenta a diversidade inicial, embora não tenha resultado em soluções substancialmente melhores.



(a) 2 casas decimais (36 bits)



(b) 5 casas decimais (76 bits)

Figura 4.12: Comparação da evolução da entropia (Shannon) por variável para diferentes precisões (2 vs 5 casas decimais).

Comparação entre os métodos de Joines et al. e Hadj-Alouane & Bean

Nesta subsecção procede-se à comparação entre duas abordagens distintas para a penalização dinâmica em algoritmos genéticos: o método proposto por Joines et al. [4] e o método de Hadj-Alouane & Bean [2]. Ambos os métodos têm como objetivo lidar com restrições em problemas de otimização, mas diferem na forma como ajustam a severidade da penalização ao longo do processo evolutivo.

O método de Joines et al. utiliza uma função de penalização baseada em parâmetros fixos que aumentam progressivamente com a geração, impondo maior pressão seletiva à medida que o algoritmo avança. Por outro lado, o método de Hadj-Alouane & Bean adota uma estratégia adaptativa, ajustando dinamicamente o parâmetro λ em função do comportamento da população, o que permite maior flexibilidade na gestão das violações.

A análise apresentada considera os melhores resultados obtidos por cada método, avaliando métricas como o valor da função objetivo, mérito, número de restrições violadas e severidade da

penalização (Tabela 4.5). Para complementar a comparação quantitativa, são discutidos os padrões de convergência e a evolução da diversidade, com base nos gráficos gerados durante as simulações.

Tabela 4.5: Comparação entre os métodos de penalização dinâmica (Joines et al.) e Hadj-Alouane & Bean.

Método	$f(x)$	**Mérito**	**Restr. violadas**	**Máx. violação**	k_{stop}
Joines et al.	2.431248	97.568495	1	0.000321	100
Hadj-Alouane & Bean	2.380363	97.615101	2	0.005381	100

A Figura 4.13 mostra a evolução do mérito segundo o método de Joines et al., evidenciando uma subida rápida nas primeiras gerações e estabilização após cerca de 30 iterações. O melhor indivíduo atingiu um mérito de 97.568495, mantendo-se praticamente constante até à geração 100, o que indica convergência precoce e baixa diversidade após a fase inicial.

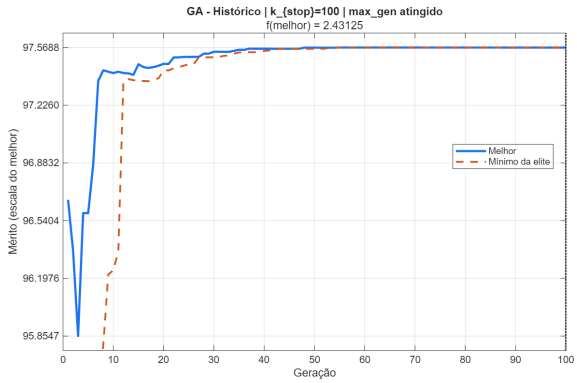


Figura 4.13: Evolução do mérito ao longo das gerações (método de Joines et al.).

De forma consistente, a Figura 4.14 apresenta a evolução da função objetivo, que iniciou acima de 3.4 e sofreu uma redução abrupta nas primeiras 10 gerações, aproximando-se do valor final $f(x) = 2.431248$ por volta da geração 20. Após este ponto, não se verificaram melhorias significativas, sugerindo estagnação do processo evolutivo.

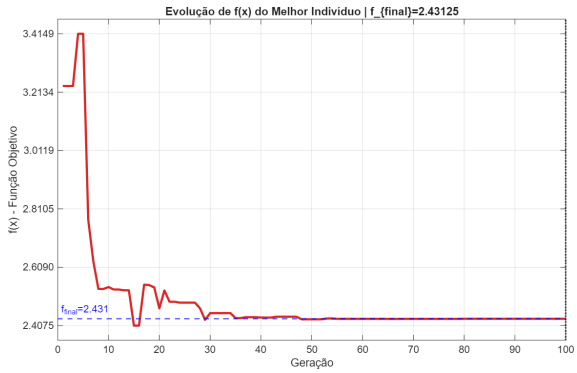


Figura 4.14: Evolução da função objetivo $f(x)$ do melhor indivíduo (método de Joines et al.).

A Figura 4.15 ilustra a dinâmica da penalização e do parâmetro λ. Observa-se que a penaliza-

ção inicial foi elevada (≈ 0.65), mas decresceu rapidamente para um valor próximo de zero antes da geração 20, enquanto $(Ck)^{\alpha}$ aumentou exponencialmente até cerca de 2500 na última geração. Este comportamento confirma a lógica do método Joines: penalização severa no início e aumento progressivo da pressão seletiva.

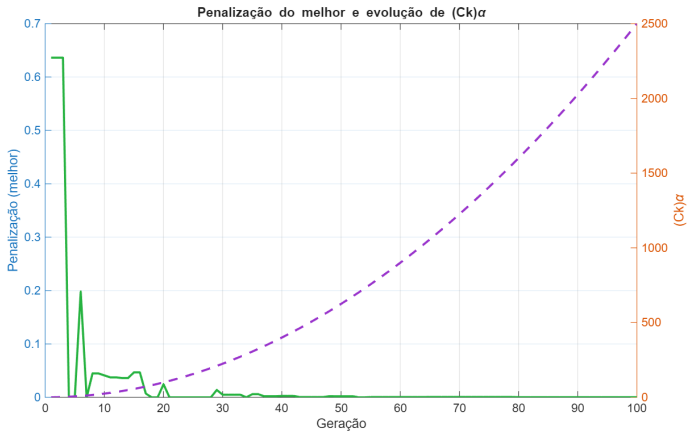


Figura 4.15: Penalização do melhor indivíduo e evolução do parâmetro $(Ck)^{\alpha}$ ao longo das gerações (método de Joines et al.).

Por fim, a Figura 4.16 evidencia a evolução da entropia por variável, com reduções acentuadas nas primeiras 20 gerações e valores próximos de zero após a geração 40. Este padrão indica perda precoce de diversidade, reforçando a convergência rápida mas com risco de exploração limitada.

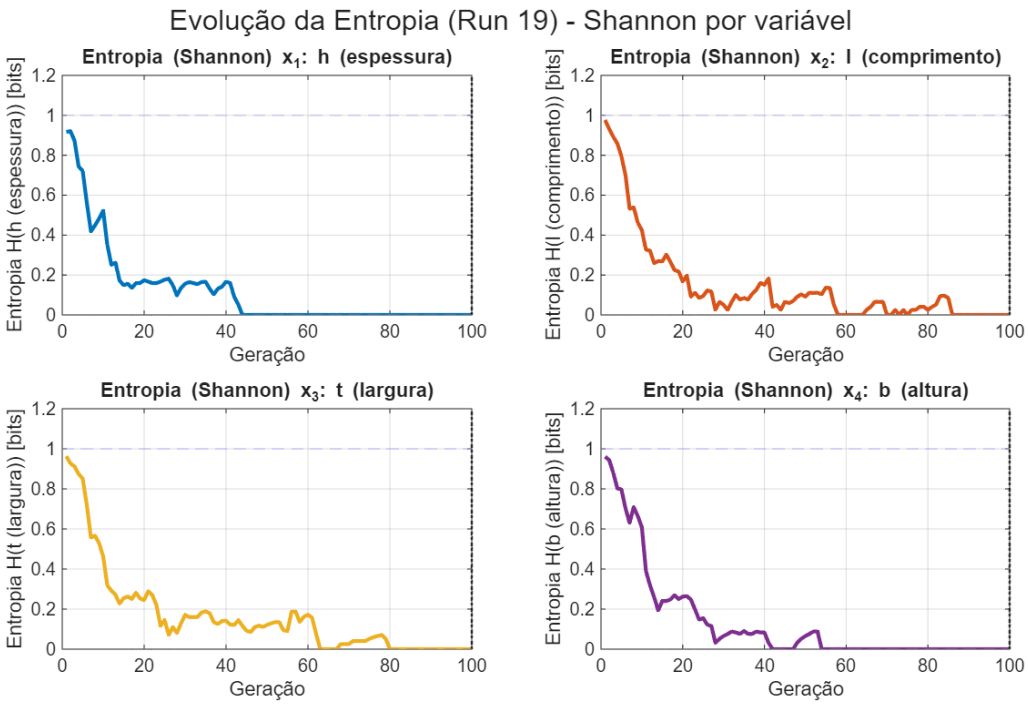


Figura 4.16: Evolução da entropia (Shannon) por variável ao longo das gerações (método de Joines et al.).

Comparando com o método de Hadj-Alouane & Bean, que apresentou melhor valor da função objetivo ($f(x) = 2.380363$) e maior mérito (97.615101), conclui-se que este último favorece soluções mais próximas do ótimo teórico, embora com ligeiras violações adicionais das restrições. Já o método de Joines et al. demonstrou maior conformidade, com menor número de restrições violadas e penalização residual, mas à custa de um valor objetivo mais elevado.

A aplicação dos algoritmos genéticos ao problema SMA7 permitiu obter soluções muito próximas da solução ótima teórica, evidenciando a capacidade destes métodos evolucionários para explorar eficazmente espaços de procura não convexos e sujeitos a múltiplas restrições. Verificou-se uma convergência rápida nas primeiras gerações, seguida de uma estabilização associada à perda gradual de diversidade, fenómeno característico deste tipo de algoritmos. A utilização de penalização dinâmica revelou-se essencial para conduzir a população para regiões viáveis sem comprometer a fase inicial de exploração. Embora persistam pequenas violações residuais das restrições e o custo computacional seja mais elevado face aos métodos determinísticos, os resultados demonstram que os algoritmos genéticos constituem uma alternativa robusta, especialmente em cenários onde a superfície de procura apresenta múltiplos mínimos locais. No capítulo seguinte procede-se à comparação direta entre ambas as abordagens.

Capítulo 5

Comparação de Resultados

A presente secção tem como objetivo comparar o desempenho dos métodos estudados, os métodos clássicos baseados em gradiente (Fletcher-Reeves e Polak-Ribière) e o algoritmo genético, segundo critérios fundamentais de avaliação: qualidade da solução obtida, tempo computacional e número de iterações ou gerações necessárias para atingir a convergência. Esta análise permite contextualizar, de forma integrada, os resultados apresentados nos capítulos anteriores e avaliar as vantagens e limitações inerentes a cada abordagem no âmbito do problema SMA7.

A Tabela 5.1 sintetiza os principais indicadores de desempenho de cada método, servindo como ponto de partida para a discussão que se segue.

Tabela 5.1: Comparação entre métodos de otimização.

Método	$f(x)$	**Tempo (s)**	**Iter./Geração**
Clássico (β_{FR})	2.381	0.0925	6
Clássico (β_{PR})	2.381	0.3874	11
Genético	2.38036	0.1416	100

5.1 Discussão dos Resultados

Ao analisar os dois métodos clássicos, observa-se uma diferença significativa no respetivo custo computacional, apesar de ambos alcançarem exatamente o mesmo valor de $f(x)$. O método de Fletcher-Reeves necessita apenas de **6 iterações** para convergir, enquanto Polak-Ribière requer **11 iterações**, apresentando um tempo computacional superior a quatro vezes o anterior. Esta discrepância resulta da forma distinta como cada método atualiza o parâmetro de conjugação β_k: o método de Fletcher-Reeves utiliza apenas a norma do gradiente atual, conduzindo a direções de pesquisa mais estáveis, enquanto Polak-Ribière incorpora a variação do gradiente entre iterações, podendo acelerar a convergência em superfícies complexas, mas também introduzir maior oscilação. No presente problema, cuja superfície de procura se torna relativamente bem condicionada após a normalização das restrições, a formulação de Fletcher-Reeves revelou ser mais eficiente.

Relativamente ao algoritmo genético, verifica-se que o seu tempo computacional é superior ao do método clássico mais eficiente, mas inferior ao do método clássico menos eficiente. Este com-

portamento é natural, uma vez que os métodos evolucionários operam sobre populações inteiras e requerem múltiplas avaliações da função por geração, o que aumenta o custo global por unidade de progresso. Contudo, este custo pode ser **substancialmente reduzido** através de critérios de paragem menos conservadores, populações de menor dimensão ou ajustando parâmetros como a elitização e a penalização dinâmica. Assim, os algoritmos genéticos oferecem maior flexibilidade computacional, permitindo que a convergência seja acelerada caso se pretenda privilegiar a eficiência temporal em detrimento da precisão.

No contexto deste trabalho, privilegiou-se a obtenção de uma solução de elevada qualidade, mesmo que tal implicasse tempos computacionais mais elevados. O foco principal residiu em demonstrar a capacidade do algoritmo genético para atingir valores de $f(x)$ praticamente coincidentes com o ótimo teórico e em analisar em detalhe o comportamento da população ao longo das gerações.

Por fim, importa salientar que o tempo computacional reportado para o algoritmo genético pode não refletir o custo real mínimo associado à execução do método. Durante a análise apresentada no Capítulo 4, foram recolhidas diversas métricas adicionais, tais como: histórico do mérito, evolução da penalização, entropia, distribuição da população, entre outras. A recolha e armazenamento destas quantidades introduz operações extra de escrita em memória e processamento, aumentando artificialmente o tempo total de execução. Assim, os tempos reportados devem ser interpretados como uma aproximação superior ao custo real, não representando exclusivamente o tempo estritamente necessário ao cálculo dos valores do algoritmo genético.

Capítulo 6

Conclusão

O estudo desenvolvido ao longo deste trabalho permitiu analisar, implementar e comparar duas abordagens distintas para a resolução do problema SMA7: os métodos clássicos baseados em gradiente e os algoritmos genéticos. A formulação matemática do problema, que envolve múltiplas restrições não lineares e uma função objetivo de natureza complexa, constituiu um cenário adequado para avaliar a eficiência, robustez e versatilidade de cada metodologia de otimização.

A aplicação dos métodos clássicos, particularmente através da estratégia do Lagrangeano Aumentado em conjunto com o Método do Gradiente Conjugado, demonstrou elevada eficácia no contexto deste problema. Ambos os esquemas de atualização do parâmetro de conjugação, Fletcher-Reeves e Polak-Ribière, convergiram para a solução ótima teórica com elevada precisão, sendo que a versão de Fletcher-Reeves apresentou o melhor desempenho computacional, alcançando o ótimo em apenas seis iterações. Estes resultados confirmam que, quando a superfície de busca é relativamente bem comportada e as restrições são devidamente normalizadas, os métodos determinísticos constituem ferramentas extremamente rápidas e eficientes.

Por outro lado, o algoritmo genético, apesar de apresentar um custo computacional superior, mostrou-se igualmente capaz de encontrar soluções muito próximas da ótima teórica. A análise detalhada do processo evolutivo revelou que a utilização de penalização dinâmica (quer através do método não-estacionário de Joines et al., quer através da abordagem adaptativa de Hadj-Alouane & Bean) desempenha um papel fundamental na condução da população para regiões viáveis do espaço de procura. As soluções obtidas apresentaram apenas pequenas violações residuais das restrições, resultado esperado devido ao número limitado de gerações e à discretização inerente à representação binária. Verificou-se ainda que o custo computacional reportado para o algoritmo genético se encontra inflacionado devido à recolha extensiva de métricas adicionais para fins de análise, não refletindo necessariamente o tempo mínimo exigido pela execução do método em modo puramente operacional.

A comparação entre os métodos permitiu concluir que, para problemas bem condicionados e com informação derivada facilmente acessível, os métodos clássicos apresentam vantagens claras em termos de velocidade e precisão. No entanto, os algoritmos genéticos demonstraram ser uma alternativa robusta, especialmente relevante em contextos onde a superfície de procura é irregular,

multimodal ou com regiões não diferenciáveis. A sua capacidade de explorar amplamente o espaço de soluções, aliada à flexibilidade na definição de critérios de paragem e parâmetros evolutivos, torna-os ferramentas valiosas em cenários onde os métodos determinísticos podem falhar ou ficar presos em ótimos locais.

Em síntese, a combinação das duas metodologias permitiu não só obter uma solução ótima para o problema em estudo, como também compreender as suas diferentes vantagens e limitações. A complementaridade entre métodos determinísticos e evolucionários reforça a importância de uma abordagem híbrida e informada na otimização de sistemas mecânicos, destacando-se que a escolha do método mais adequado deve ser sempre guiada pela natureza específica do problema e pelos objetivos do processo de otimização.

Referências Bibliográficas

[1] C. C. António. *Optimization of Mechanical Systems - Fundamentals and algorithms*. 2024.

[2] A. Hadj-Alouane and J. C. Bean. A genetic algorithm for the multiple-choice integer program. *Computers & Operations Research*, 22(1):57–67, 1995.

[3] F. Herrera, M. Lozano, and J. L. Verdegay. Tackling real-coded genetic algorithms: Operators and tools for behavioural analysis. In *Artificial Intelligence Review*, volume 12, pages 265–319. Springer, 1998.

[4] J. Joines and C. Houck. On the use of non-stationary penalty functions to solve constrained optimization problems with genetic algorithms. In *Proceedings of the 1st IEEE Conference on Evolutionary Computation*, pages 579–584, 1994.

[5] S. S. Rao. *Engineering Optimization Theory and Practice || Introduction to Optimization*. John Wiley Sons., 2019.